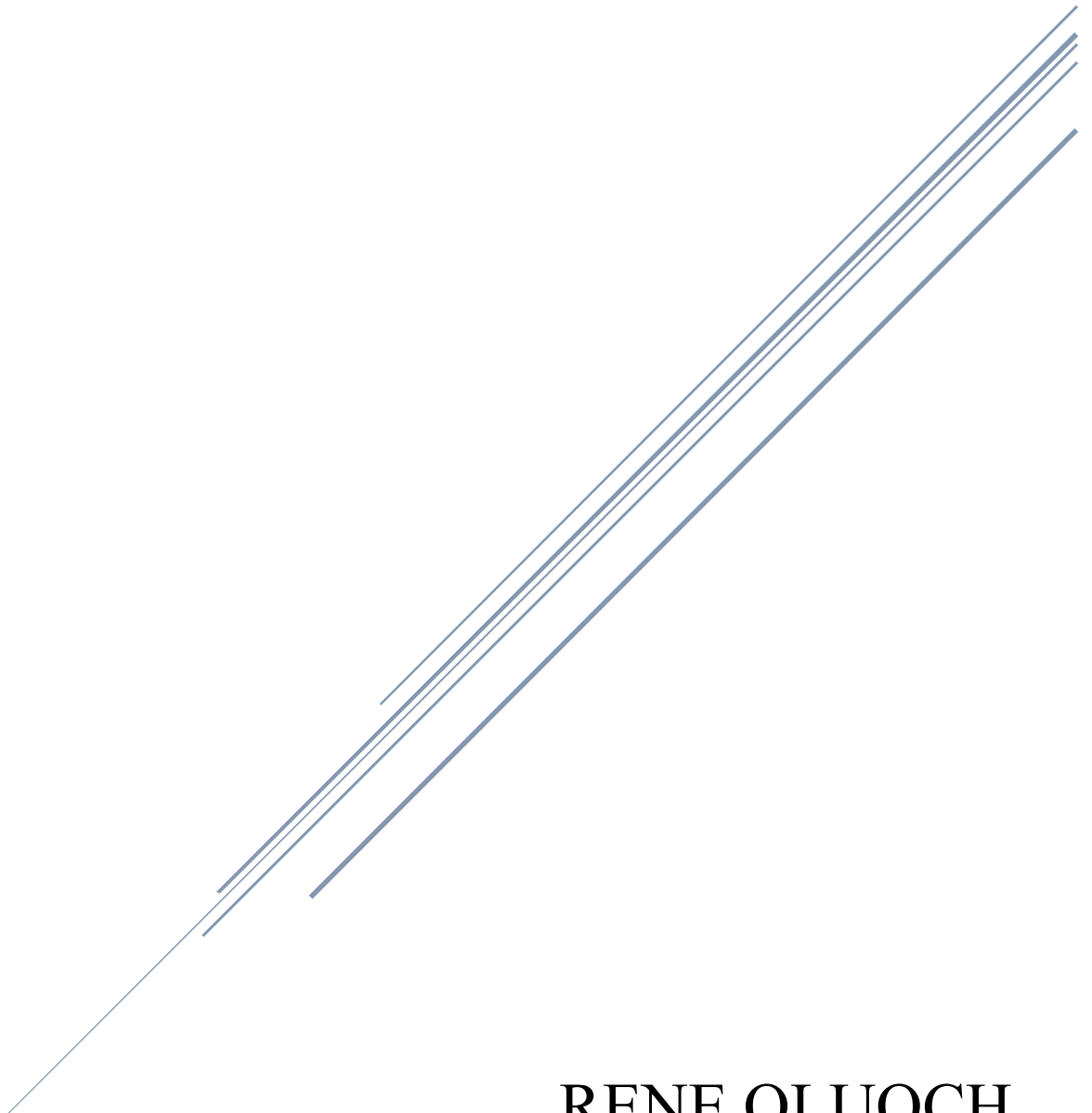


KEY VAULT (IMPLEMENTING SECURE DATA BY SETTING UP ALWAYS ENCRYPTED)

AZ500 Lab 10



RENE OLUOCH
CS-CNS09-25030

Contents

Introduction.....	2
Exercise 1 Deploying the base infrastructure from an ARM template	2
Task 1: Deploying Base Infrastructure	2
Exercise 2 Configuring the Key Vault resource with a key and a secret	4
Task 1: Creating an Azure Key Vault.....	4
Task 2: Registering a Client Application in Microsoft Entra ID	8
Task 3: Granting Application Access to Key Vault.....	11
Exercise 3: Configuring an Azure SQL database and a data-driven application	14
Task 1: Enabling a client application to access the Azure SQL Database service.....	14
Task 2: Creating a Policy Allowing the Application Access to the Key Vault.....	17
Task 3: Retrieving SQL Azure Database ADO.NET Connection String	19
Task 4: Logging on to the Azure VM Running Visual Studio 2019 and SQL Management Studio 19.....	21
Task 5: Creating a Table in the SQL Database and Select Data Columns for Encryption.....	21
Exercise 4 : Demonstrating the use of Azure Key Vault in encrypting the Azure SQL database .	29
Task 1: Running a data-driven application to demonstrate the use of Azure Key Vault in encrypting the Azure SQL database.....	29
Conclusion	36

Introduction

In this lab, I was tasked with creating a secure proof-of-concept application using **Azure SQL Database** and **Always Encrypted**, with all keys and secrets managed through **Azure Key Vault**. To improve security, I also registered the application in **Microsoft Entra ID** and granted it the required access. My objective was to demonstrate how to deploy infrastructure using an ARM template, configure Key Vault, secure SQL data columns via Always Encrypted, and build a client application that utilizes Key Vault for encryption key operations—all within the **East US** region.

Exercise 1 Deploying the base infrastructure from an ARM template

Task 1: Deploying Base Infrastructure

To begin, I signed into the Azure portal using an account with Contributor permissions and navigated to **Deploy a custom template**. I uploaded the provided ARM template file, **az-500-10_azuredeploy.json**, reviewed its contents to confirm it would deploy a VM (with Visual Studio 2019 and SSMS) and an Azure SQL Database named *medical*, and then initiated the deployment.

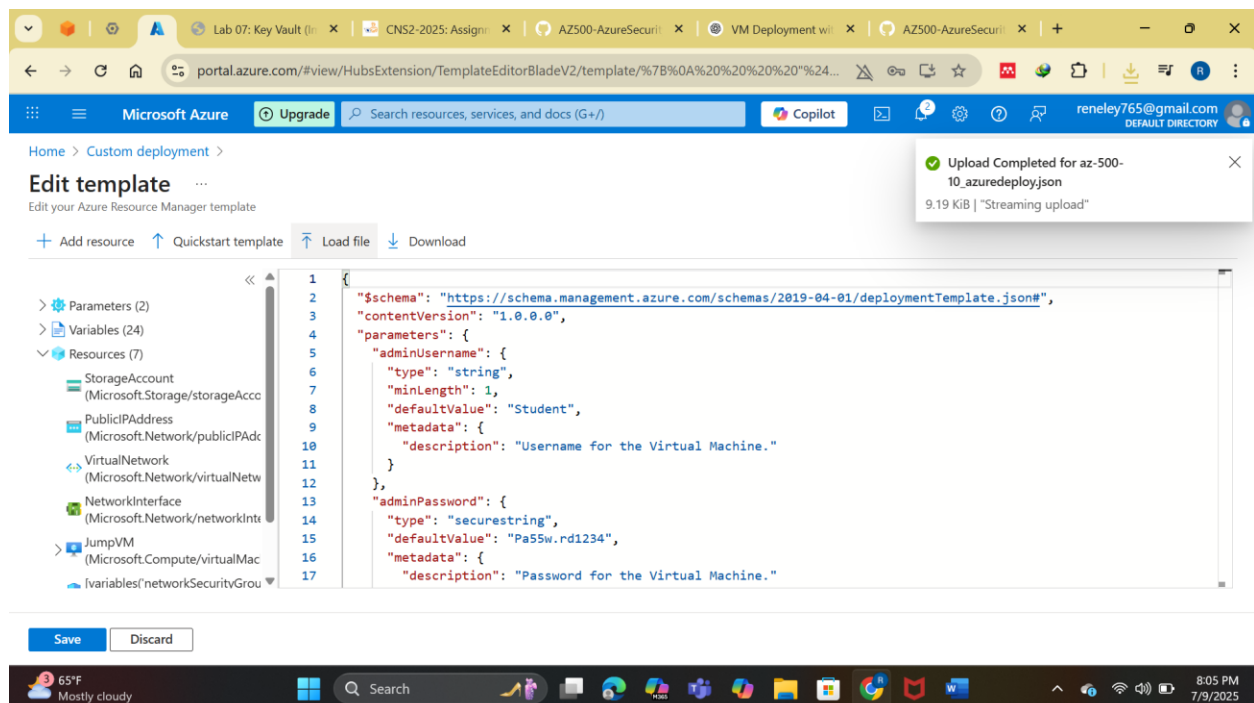


Fig 1.0 Loading the json file

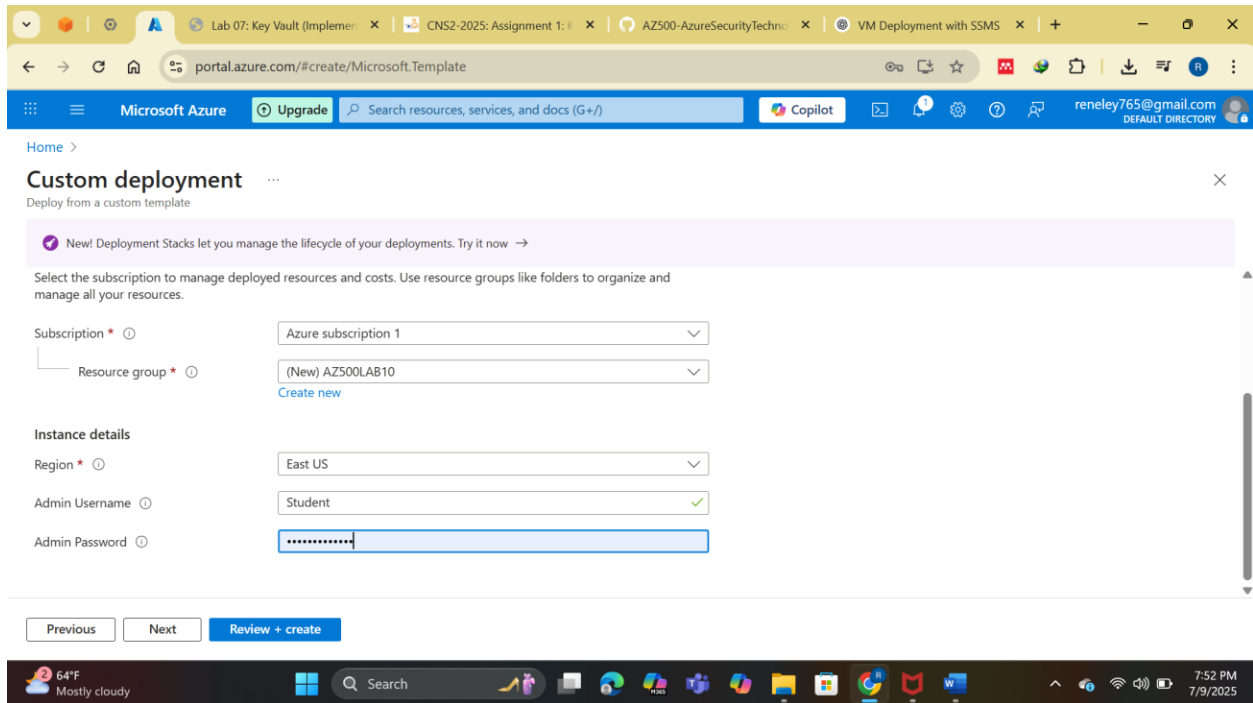


Fig 1.1 Deploying az-500-10_azuredeploy.json and creating the resource

I created a resource group named **AZ500LAB10**, chose the **East US** region, and specified **Student** for the admin username along with my secure password. After validating the settings, I clicked **Create**. Also, I updated the json file as the file given for the lab use deployed with error(validation failed. It had outdated image resources) and was successful after updating

The ARM template failed because (1) the requested **VM image or size** was not valid or available in your region, (2) the chosen VM size (Standard_DS2_V2) had no capacity in East US, (3) my subscription quota for **Basic SKU Public IP addresses** is zero, blocking new IPv4 Basic Public IPs, and (4) the SQL Server deployment failed likely due to the admin password not meeting Azure SQL's strong password requirements. To fix these: use a valid **Windows Server 2022 Datacenter** image, switch the VM size to an available option like Standard_D2s_v3, set the **Public IP SKU to Standard** instead of Basic to bypass quota limits, and ensure your SQL admin password is long and complex enough to pass policy. Also I changed the sql region to eastus 2 as the ARM template could not deploy in east us due to availability issues.

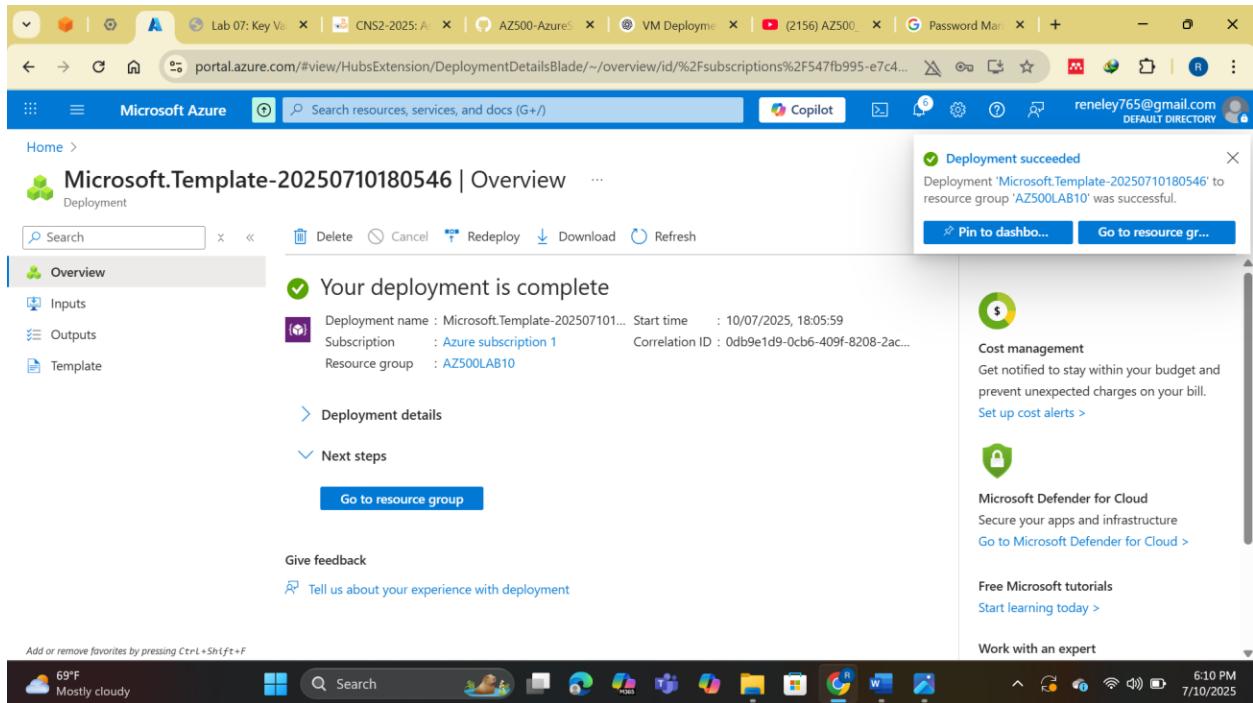


Fig 1.2 Successful creation of the resource

Exercise 2 Configuring the Key Vault resource with a key and a secret

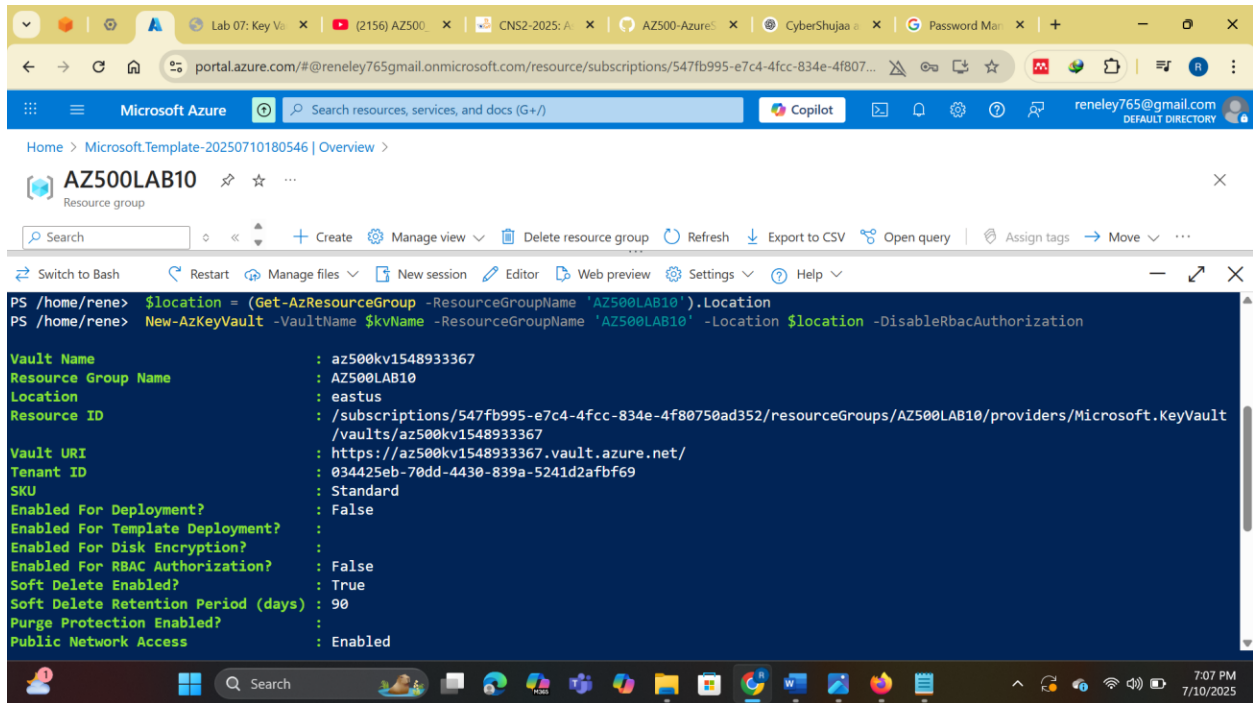
Task 1: Creating an Azure Key Vault

To begin securing data in Azure, I first created a new **Azure Key Vault** and configured its permissions to ensure only authorized operations are allowed. I started by opening **Cloud Shell** in the Azure portal by clicking the icon at the top right next to the search bar. When prompted, I selected **PowerShell** and created the required storage if it wasn't already set up. I double-checked that **PowerShell** was selected in the drop-down menu at the top of the Cloud Shell pane so I could run my commands properly.

Inside the Cloud Shell, I ran a simple script to create a unique name for my Key Vault by typing: `$kvName = 'az500kv' + $(Get-Random)`.

Next, I set the location for the Key Vault to match the region of my resource group by running: `$location = (Get-AzResourceGroup -ResourceGroupName 'AZ500LAB10').Location`.

Once I had the name and location ready, I created the Key Vault itself by executing: `New-AzKeyVault -VaultName $kvName -ResourceGroupName 'AZ500LAB10' -Location $location -DisableRbacAuthorization`.



```
PS /home/rene> $location = (Get-AzResourceGroup -ResourceGroupName 'AZ500LAB10').Location
PS /home/rene> New-AzKeyVault -VaultName $kvName -ResourceGroupName 'AZ500LAB10' -Location $location -DisableRbacAuthorization

Vault Name                : az500kv1548933367
Resource Group Name       : AZ500LAB10
Location                   : eastus
Resource ID                : /subscriptions/547fb995-e7c4-4fcc-834e-4f80750ad352/resourceGroups/AZ500LAB10/providers/Microsoft.KeyVault/vaults/az500kv1548933367
Vault URI                  : https://az500kv1548933367.vault.azure.net/
Tenant ID                  : 034425eb-70dd-4430-839a-5241d2afb69
SKU                         : Standard
Enabled For Deployment?    : False
Enabled For Template Deployment? :
Enabled For Disk Encryption? :
Enabled For RBAC Authorization? : False
Soft Delete Enabled?       : True
Soft Delete Retention Period (days) : 90
Purge Protection Enabled?  :
Public Network Access      : Enabled
```

Fig 1.3 Output of the powershell command

After running this command, I made sure to note down the **Vault name** and the **Vault URI**, which would be in the format `https://az500kv1548933367.vault.azure.net/`. I closed the Cloud Shell pane once the deployment completed.

Back in the Azure portal, I used the search bar to look for **Resource groups** and pressed Enter. In the list, I clicked on **AZ500LAB10** to open it. From there, I located the Key Vault I just deployed and clicked on it to open its blade.

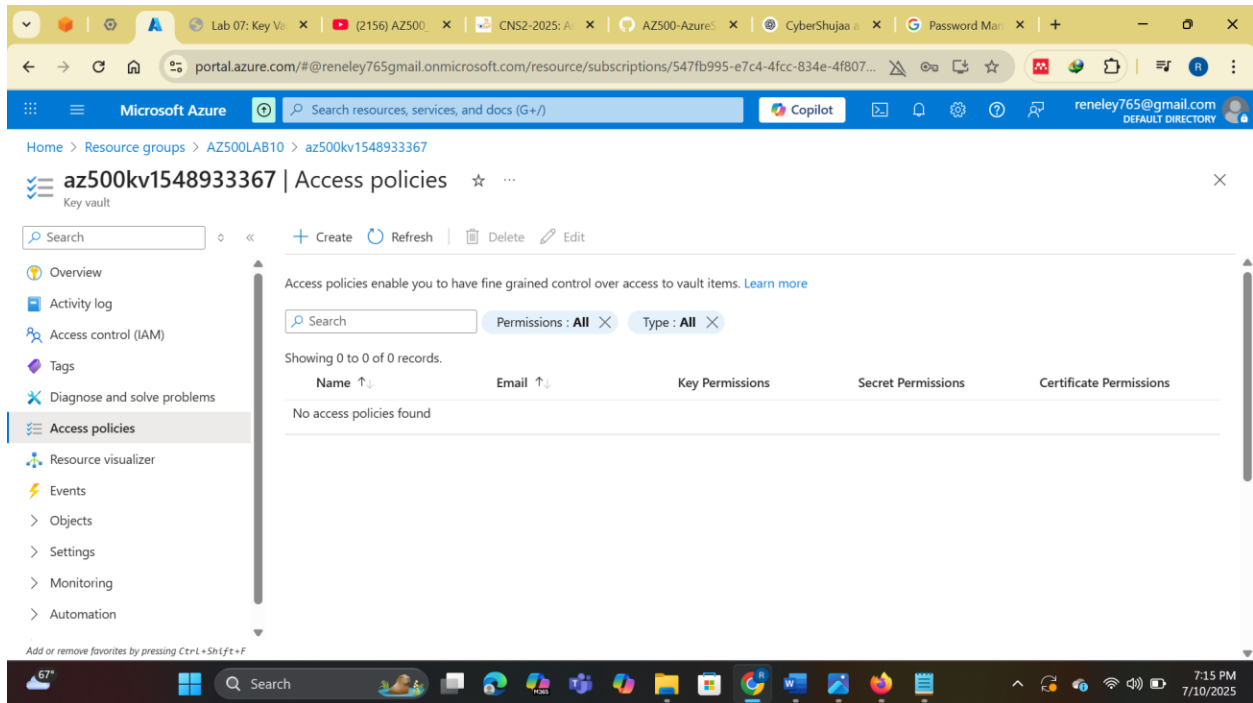


Fig 1.4 Opening the access policies of the key vault

Inside the Key Vault's **Overview** section, I selected **Access Policies** and then clicked **+ Create** to define a new access policy. On the **Create an access policy** blade, I chose the **Key, Secret, & Certificate Management** template, which automatically selected typical permissions for managing keys, secrets, and certificates. I made sure to click **Select all** under **Key permissions** to grant all nine available key permissions, then I added the **Sign** permission under **Cryptographic Operations**. For **Secret permissions**, I also clicked **Select**, and I did the same for **Certificate permissions** to grant all fifteen certificate management permissions.

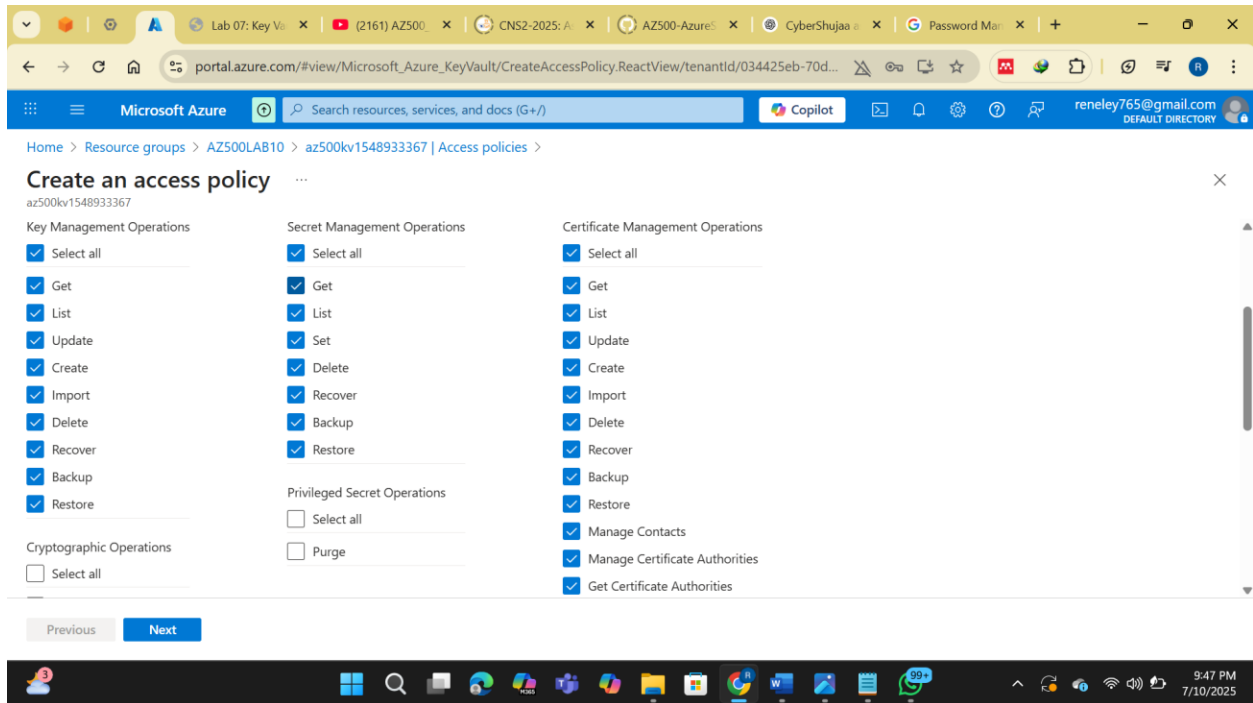


Fig 1.5 Selecting the required permissions

To apply this new policy to my account, I clicked **Select principal**, searched for my user account on the Principal blade, selected it, and clicked **Next**. I skipped the **Application** option by clicking **Next** again, then reviewed all my settings and clicked **Create** to finalize the policy.

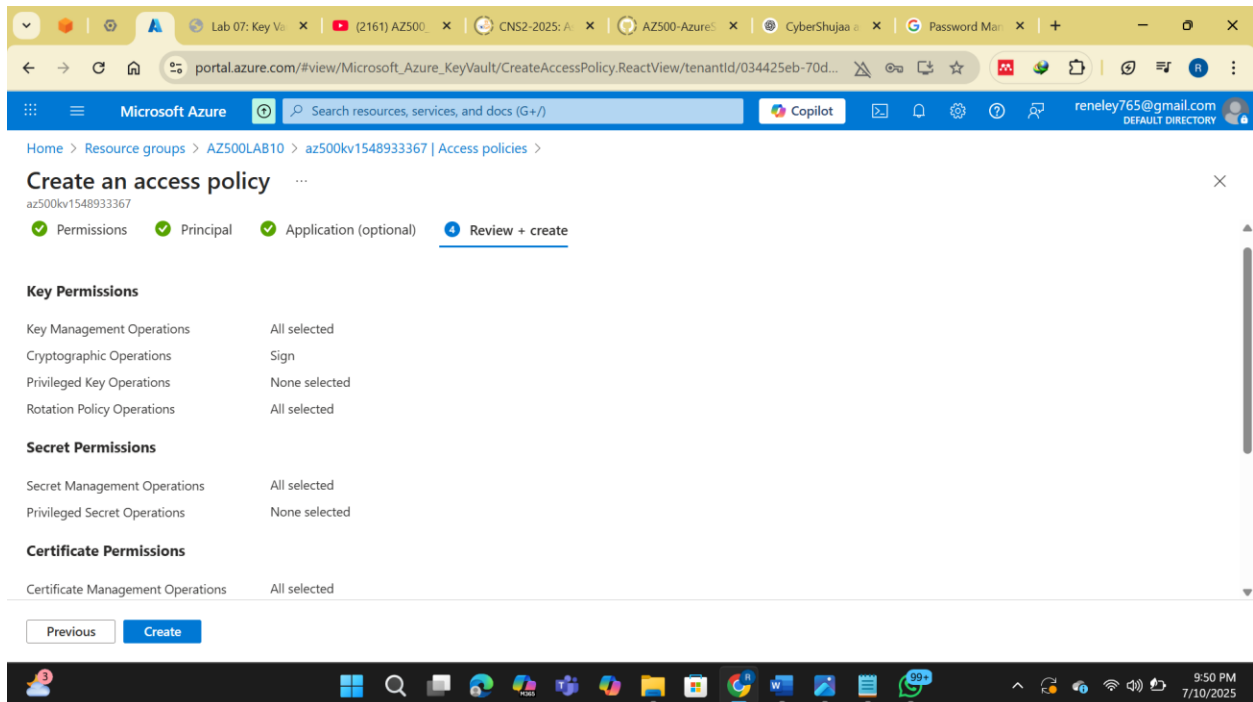


Fig 1.6 Reviewing and creating the policies

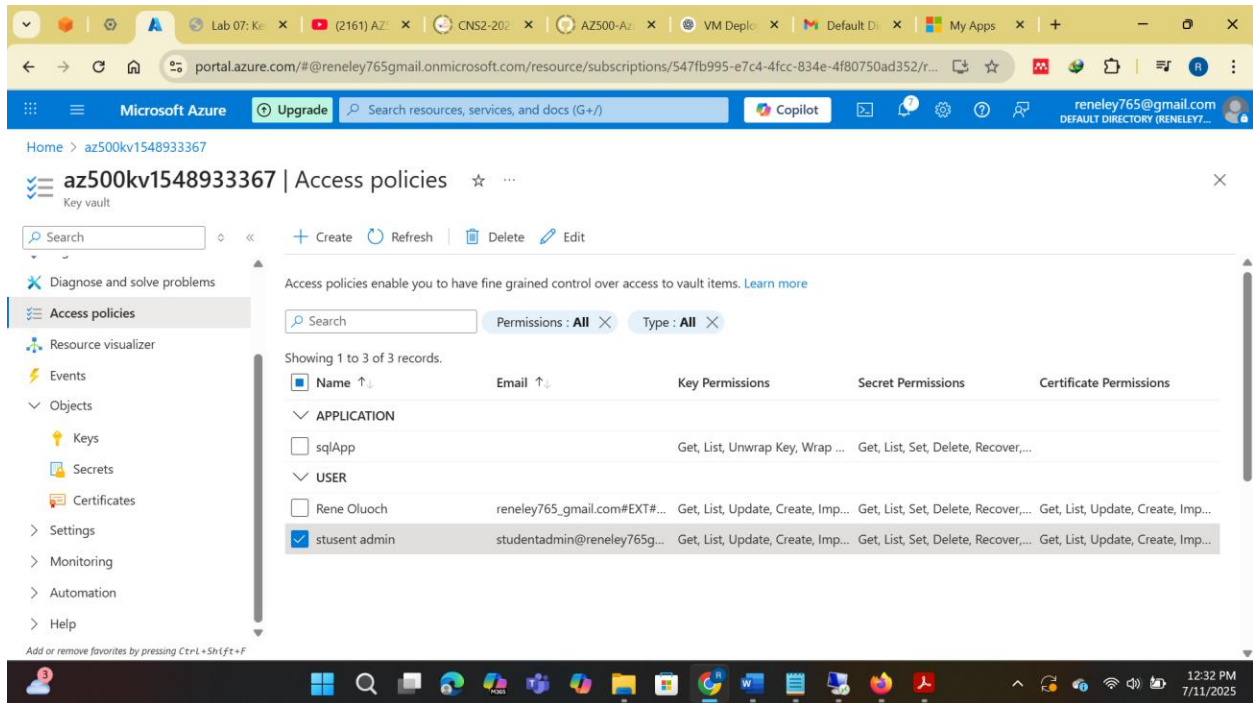


Fig 1.7 Successfully creating the key vault

At this point, my Key Vault was successfully created with the right permissions for managing keys, secrets, and certificates — making it ready to protect sensitive data securely and control access effectively.

Task 2: Registering a Client Application in Microsoft Entra ID

Next, I needed to add a cryptographic key to my Azure Key Vault and confirm that it was created properly. To do this, I went back to the **Cloud Shell** in the Azure portal and made sure that **PowerShell** was still selected in the drop-down menu in the upper-left corner of the Cloud Shell pane.

Inside the PowerShell session, I first retrieved the details of my Key Vault by running:
`$kv = Get-AzKeyVault -ResourceGroupName 'AZ500LAB10'.`

Once I had the Key Vault object stored, I added a new **software-protected key** to it by executing:

```
$key = Add-AZKeyVaultKey -VaultName $kv.VaultName -Name 'MyLabKey' -Destination 'Software'.
```

This created a key named **MyLabKey**, which I would use later in my secure applications.

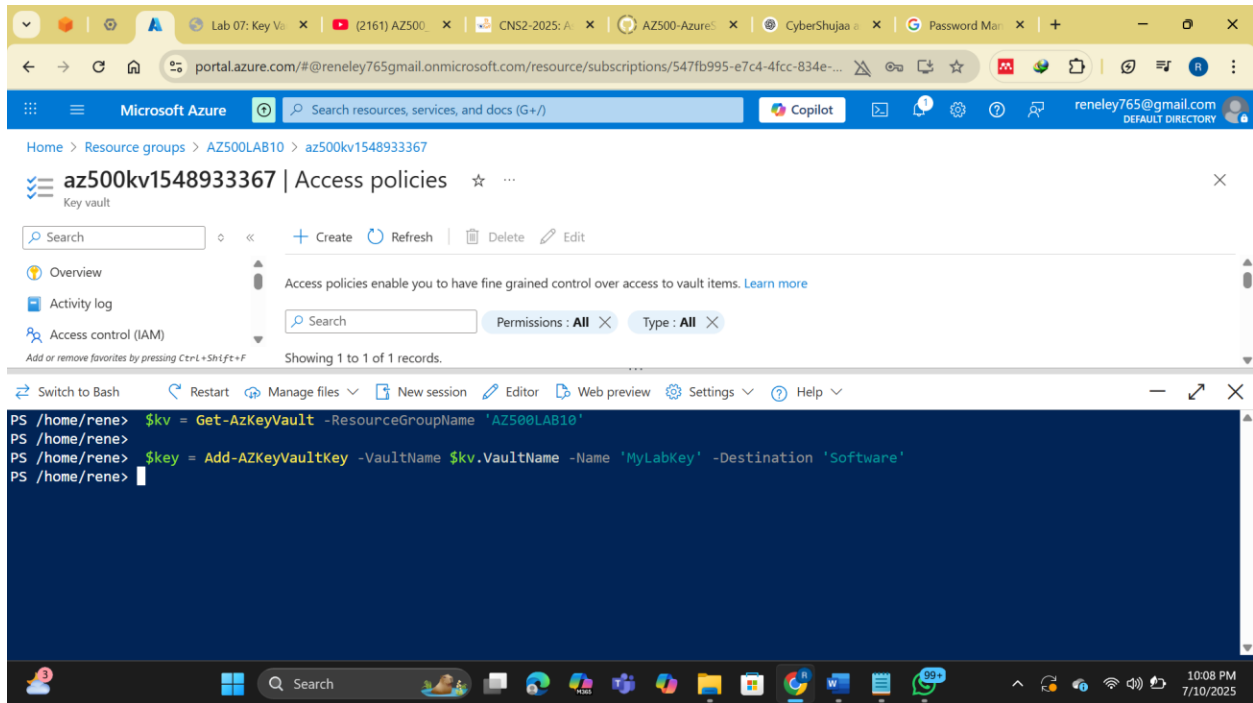


Fig 1.8 Creating MyLabKey in the cli

To make sure the key was actually created, I verified it by listing the keys in the vault with: `Get-AzKeyVaultKey -VaultName $kv.VaultName`.

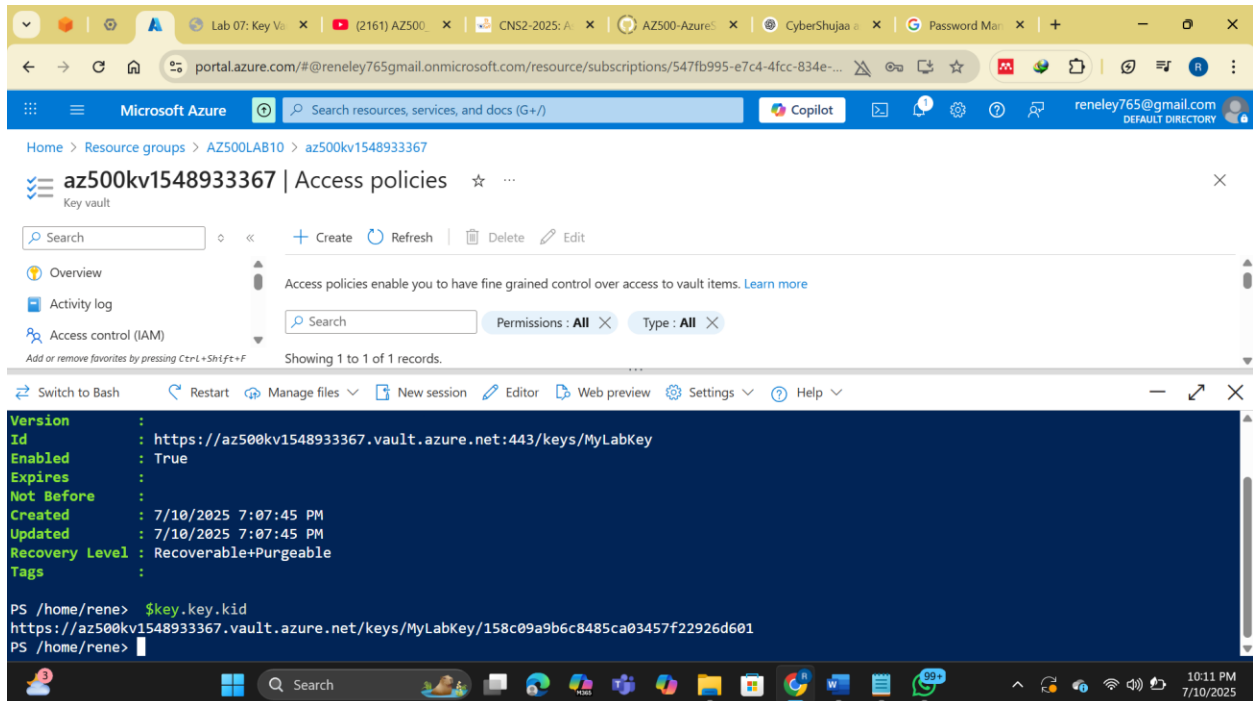


Fig 1.9 Verifying key creation

To check the full identifier for my new key, which I'd need if I wanted to reference this key programmatically, I displayed the key's unique ID by typing:
`$key.key.kid`.

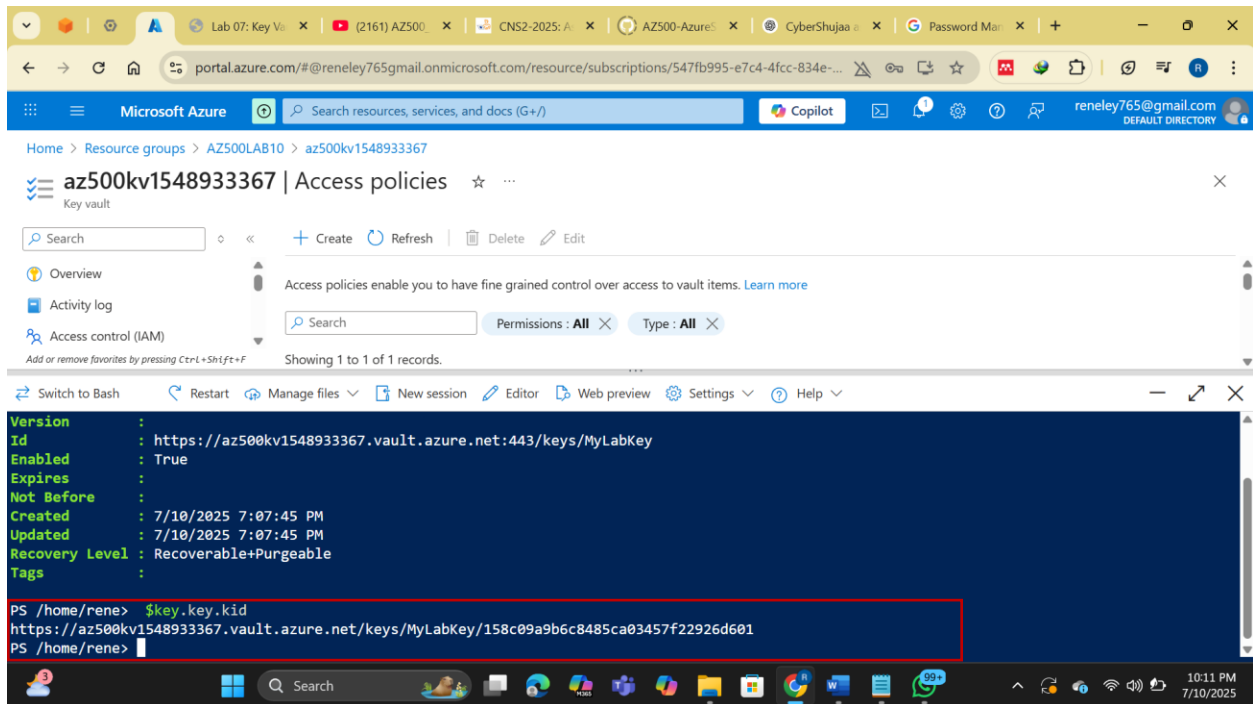


Fig 2.0 Checking full identifier for the new key

After confirming the key details in Cloud Shell, I minimized the Cloud Shell pane and went back to the Azure portal. On my **Key Vault blade**, under the **Objects** section, I clicked **Keys** to see the list of keys stored in my vault. I found **MyLabKey** in the list and clicked on it. Then, on the **MyLabKey blade**, I clicked on the entry for its current version to view all its details and settings.

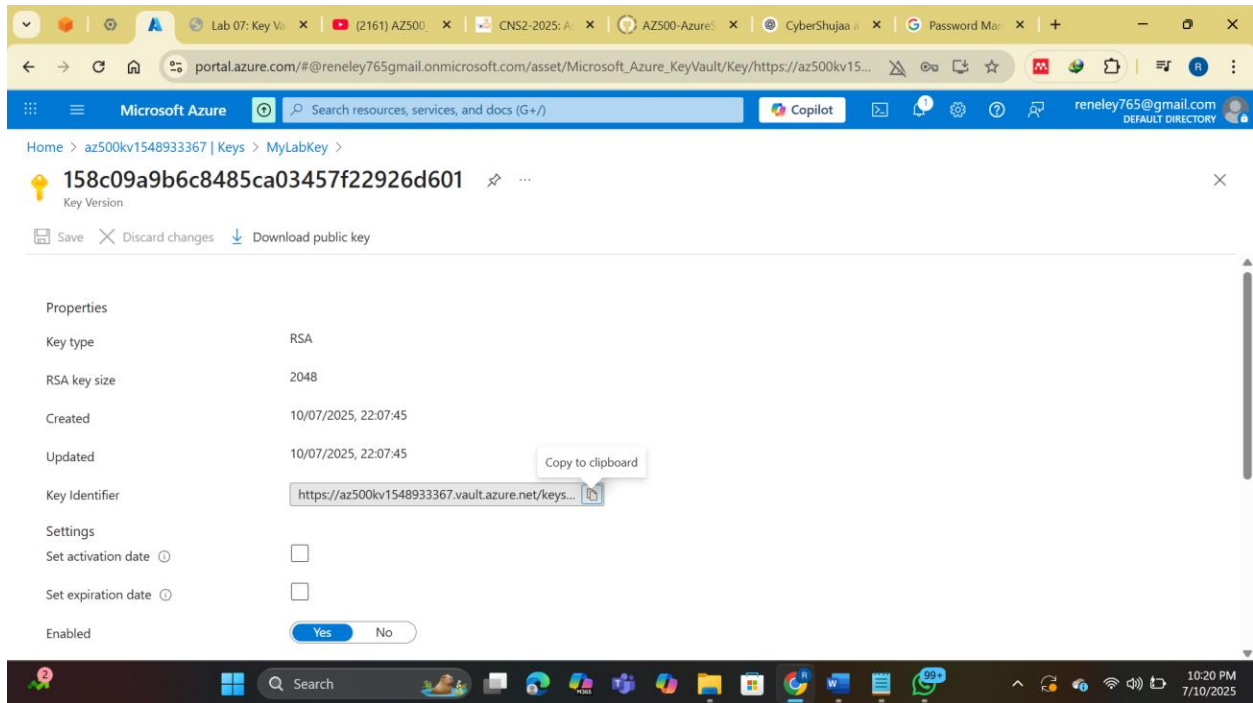


Fig 2.1 Examining information about the key

At this point, I reviewed the key information, noting the key identifier and understanding that I could always reference this key using its URL in the format
`https://az500kv1548933367.vault.azure.net/keys/MyLabKey/158c09a9b6c8485ca03457f22926d601`

By adding this key, I ensured that my Key Vault could now securely manage cryptographic operations for encrypting and decrypting sensitive data in my Azure environment.

Task 3: Granting Application Access to Key Vault

After creating and verifying my key, I moved on to storing a secret in my Azure Key Vault. I switched back to the **Cloud Shell** pane in the Azure portal and made sure **PowerShell** was still selected.

First, I needed to create a variable to hold my secure string. So, in the PowerShell session, I typed:

```
$secretvalue = ConvertTo-SecureString 'Pa55w.rd1234' -AsPlainText -Force.
```

This command safely converted the plain text password **Pa55w.rd1234** into a secure string that could be stored securely in my Key Vault.

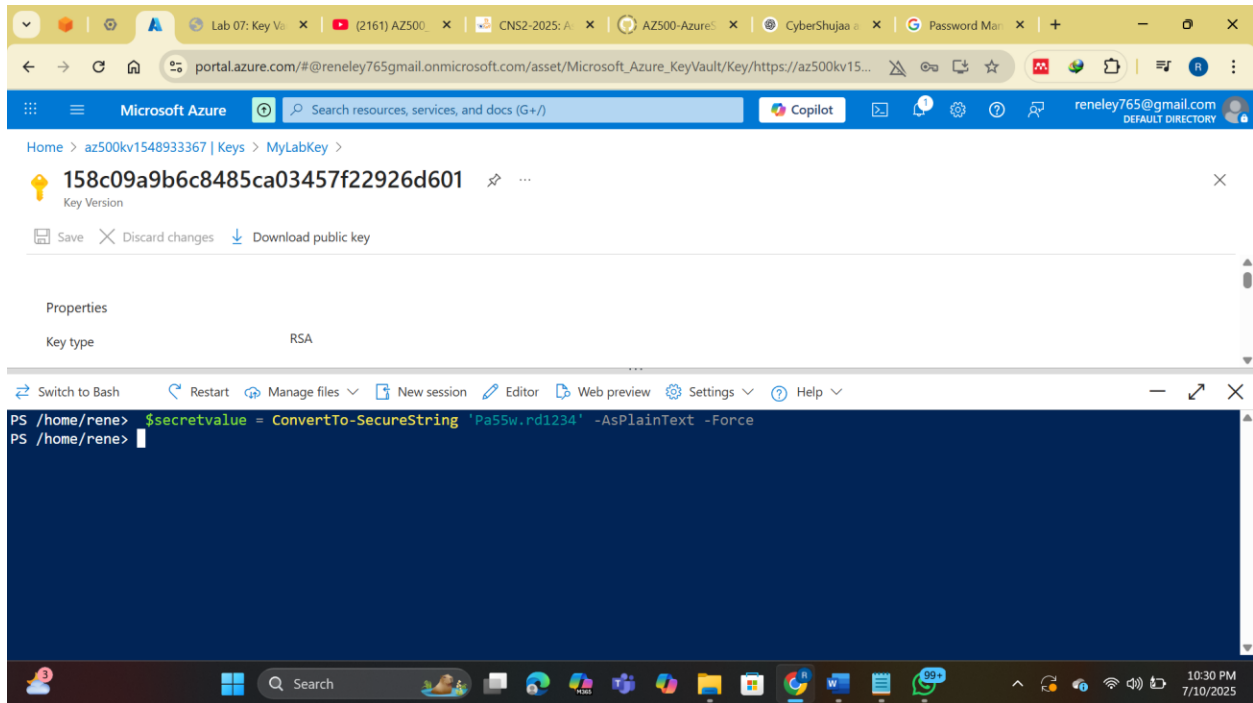


Fig 2.2 Creating a variable

Next, I added this secret into my Key Vault by running:

`$secret = Set-AZKeyVaultSecret -VaultName $kv.VaultName -Name 'SQLPassword' -SecretValue $secretvalue.`

This created a new secret named **SQLPassword**.

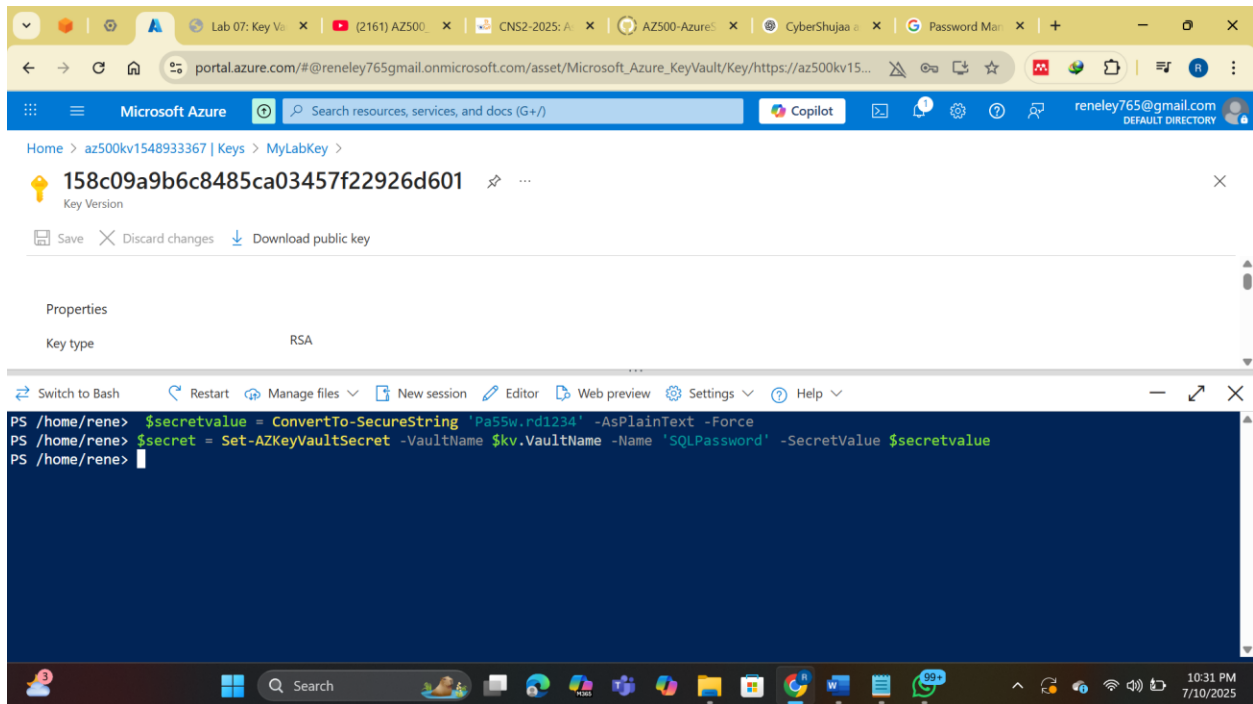
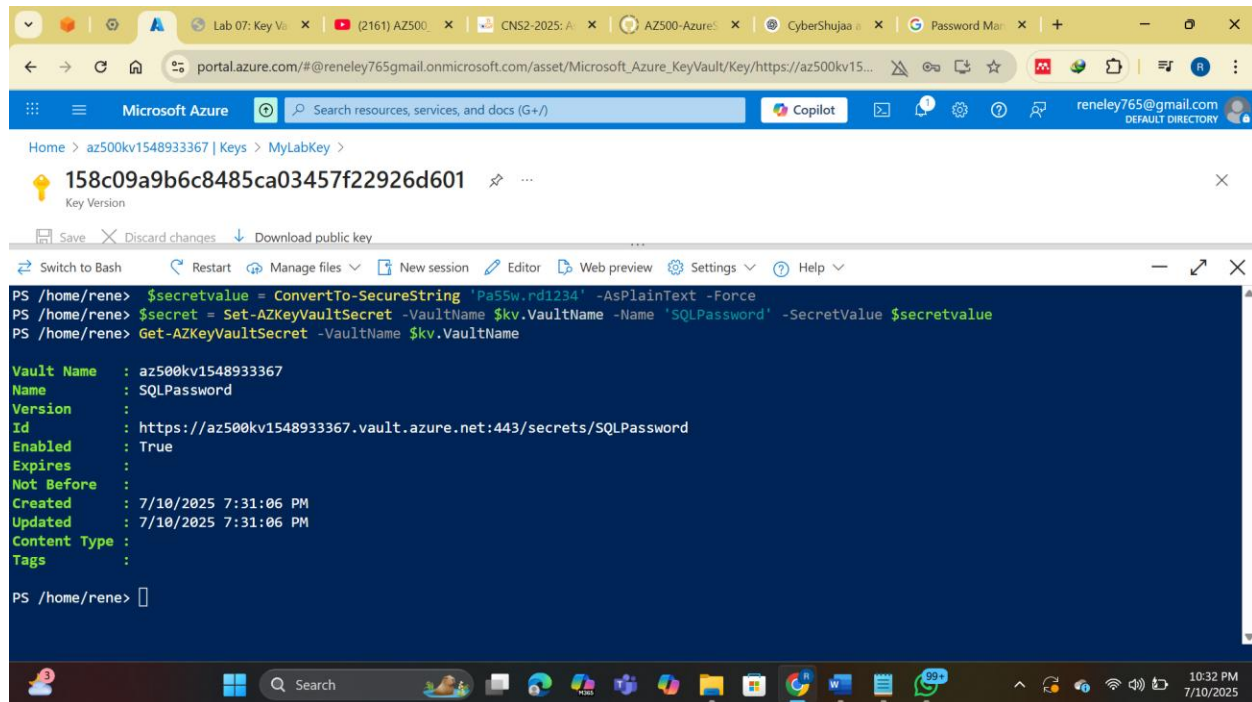


Fig 2.3 Adding secret value to the key

To confirm that my secret was stored properly, I listed all the secrets in my vault with:
`Get-AZKeyVaultSecret -VaultName $kv.VaultName`.



The screenshot shows the Azure portal interface. At the top, the breadcrumb navigation indicates the path: Home > az500kv1548933367 | Keys > MyLabKey >. Below this, a key is displayed with the name '158c09a9b6c8485ca03457f22926d601' and its version 'Key Version'. Below the key details, there is a terminal window titled 'Switch to Bash' with the following PowerShell commands and output:

```
PS /home/rene> $secretvalue = ConvertTo-SecureString 'Pa55w.rd1234' -AsPlainText -Force
PS /home/rene> $secret = Set-AZKeyVaultSecret -VaultName $kv.VaultName -Name 'SQLPassword' -SecretValue $secretvalue
PS /home/rene> Get-AZKeyVaultSecret -VaultName $kv.VaultName

Vault Name : az500kv1548933367
Name       : SQLPassword
Version    :
Id         : https://az500kv1548933367.vault.azure.net:443/secrets/SQLPassword
Enabled    : True
Expires    :
Not Before :
Created    : 7/10/2025 7:31:06 PM
Updated    : 7/10/2025 7:31:06 PM
Content Type :
Tags       :
```

Fig 2.4 Verifying that the secret is stored properly

Once I verified that the secret appeared in the list, I minimized the Cloud Shell pane and went back to the **Key Vault blade** in the Azure portal. In the **Objects** section, I clicked on **Secrets** to view all the secrets stored in my vault. There, I found **SQLPassword** in the list and clicked on it. Then, on the **SQLPassword blade**, I clicked on the entry representing its current version to see the details.

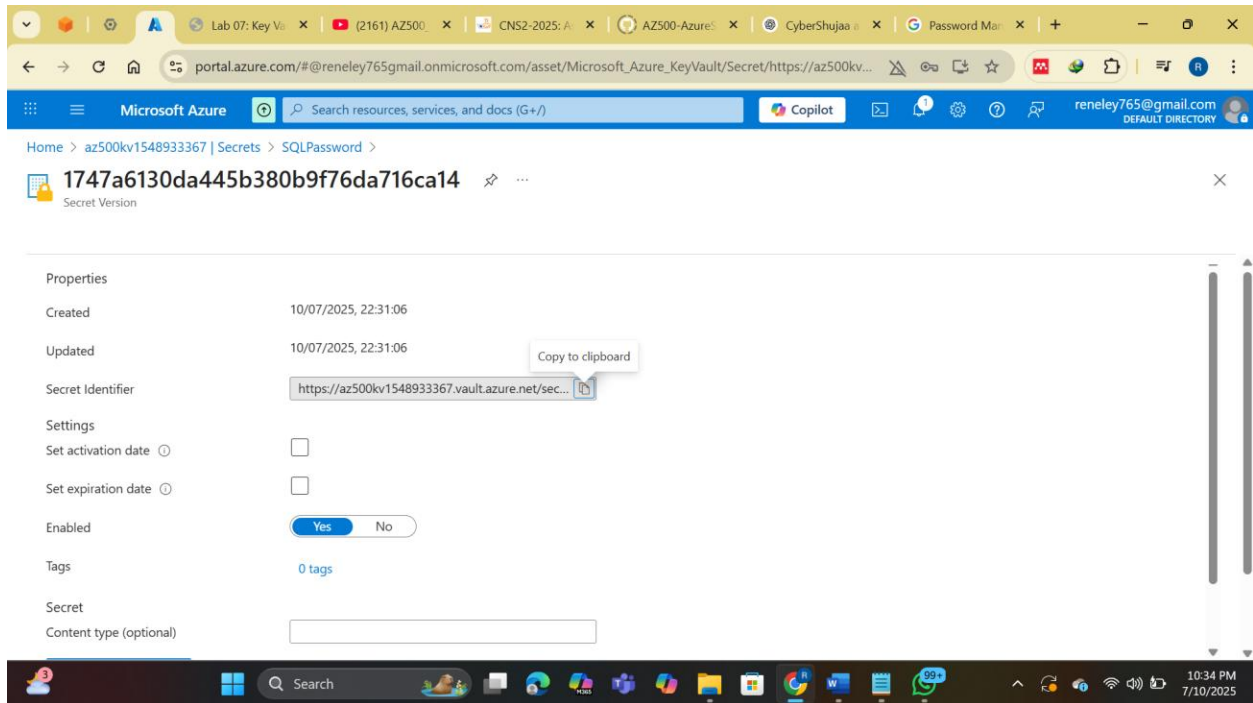


Fig 2.5 Reviewing information about my secret

I reviewed the information about my secret and took note that, similar to keys, I could always reference the most current version of this secret using its URL in the format

`https://az500kv1548933367.vault.azure.net/secrets/SQLPassword/1747a6130da445b380b9f76da716ca14`

By adding this secret, I ensured that sensitive data like passwords could be securely managed and accessed only by trusted applications or users through proper policies.

Exercise 3: Configuring an Azure SQL database and a data-driven application

Task 1: Enabling a client application to access the Azure SQL Database service.

To begin configuring my Azure SQL Database to securely interact with a client application, I first needed to register the application in **Azure Active Directory** so it could authenticate properly.

In the **Azure portal**, I went to the **Search resources, services, and docs** box at the top and typed **App Registrations**, then pressed **Enter** to open the **App Registrations** blade.

On this blade, I clicked + **New registration** to create a new application registration. On the **Register an application** page, I entered **sqlApp** as the **Name** and, for the **Redirect URI (optional)**, I selected **Web** and typed **https://sqlapp**. I left all other fields with their default values and clicked **Register** to complete the app registration.

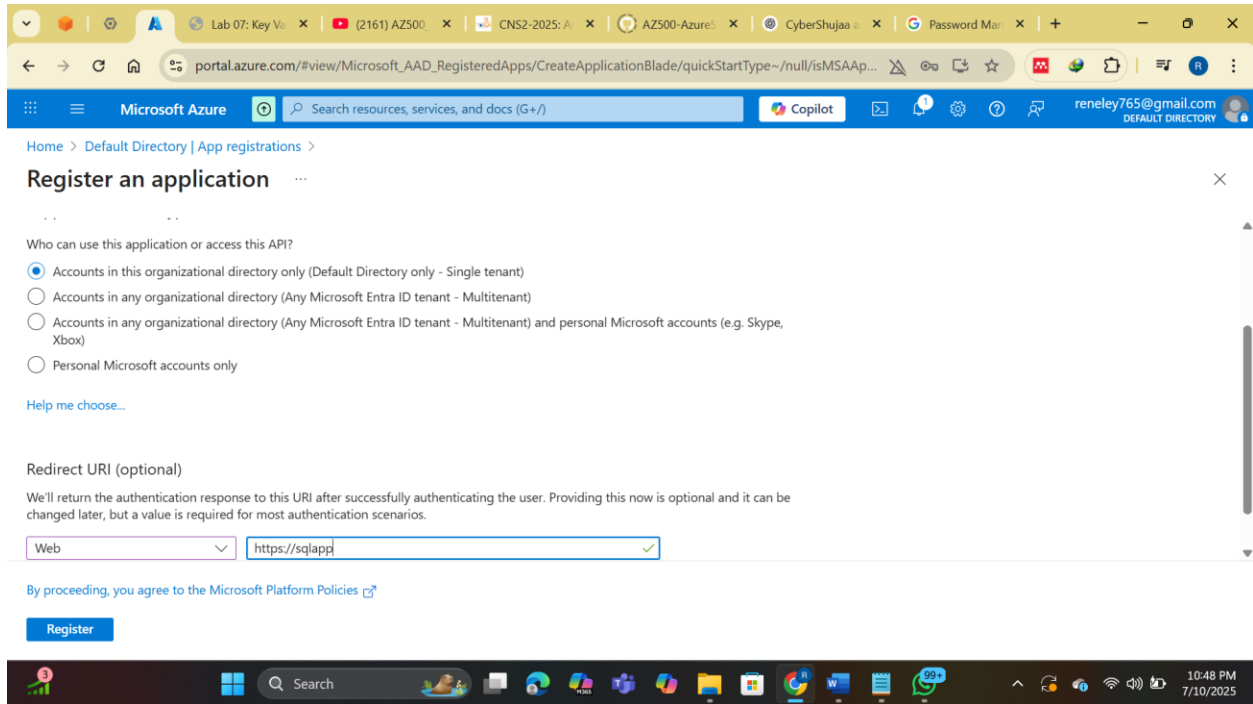


Fig 2.6 Creating an sqlapp registration

Once the registration finished, I was automatically redirected to the **sqlApp blade**, where I noted down the **Application (client) ID** shown at the top. I made sure to record this ID because I would need it later when configuring access policies and connection strings.

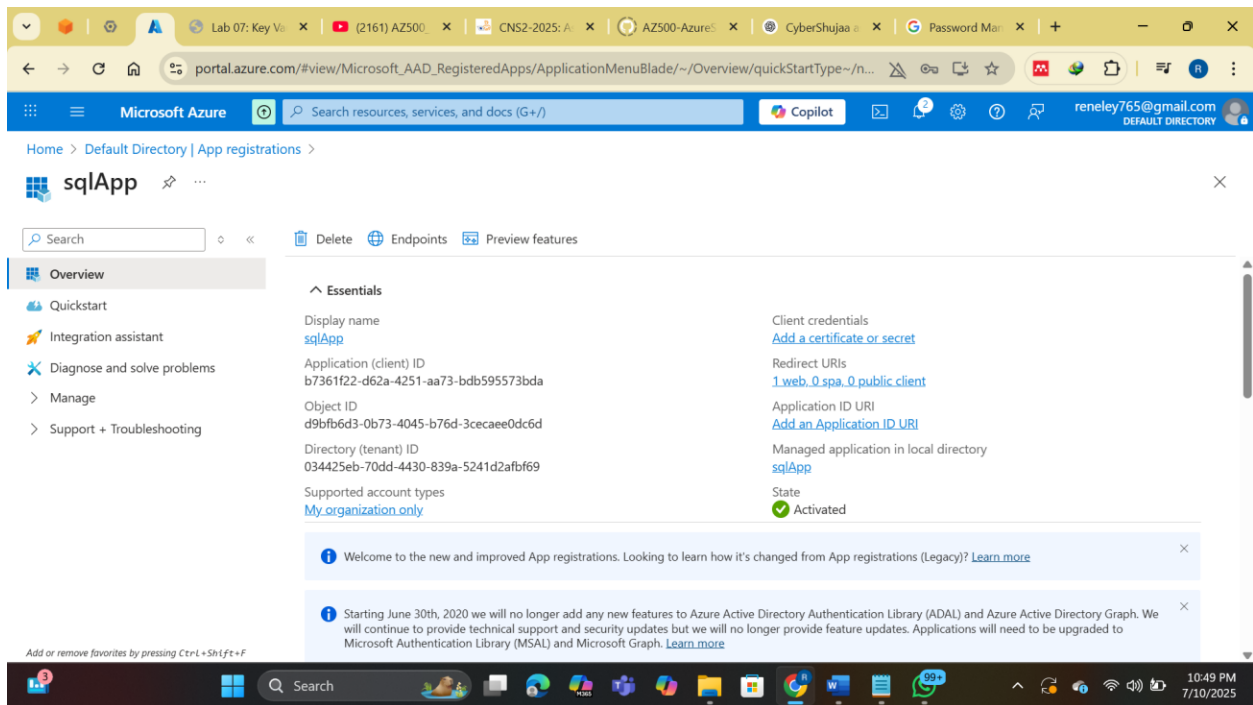


Fig 2.7 Sqlqpp blade

Next, still on the **sqlApp** blade, under the **Manage** section, I clicked **Certificates & secrets**. Here, under the **Client secrets** section, I clicked **+ New client secret** to create a new secret for my application.

In the **Add a client secret** pane that appeared, I gave the secret a description by typing **Key1** and set it to **expire in 12 months**. Then, I clicked **Add** to create the secret.

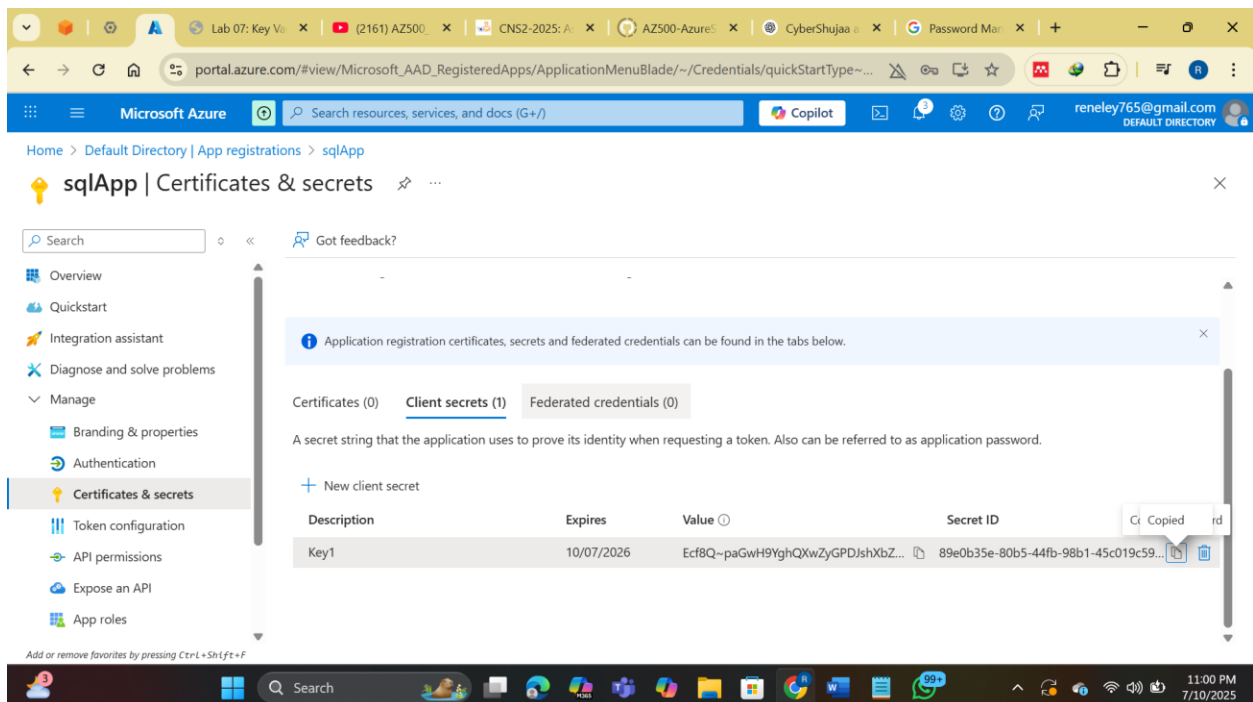


Fig 2.8 Creating a client secret (for sqlapp)

After the secret was generated, I noted down the **value of Key1**. By registering the **sqlApp** application and generating its **Application ID** and **client secret**, I set up the credentials that my client application will use to authenticate itself securely when connecting to the Azure SQL Database service.

Task 2: Creating a Policy Allowing the Application Access to the Key Vault

After registering my client application, I needed to grant it the right permissions to securely access secrets stored in my Azure Key Vault. To do this, I switched to the **Azure portal** and opened a **PowerShell session** in the **Cloud Shell pane** at the top right of the portal. I made sure **PowerShell** was selected from the drop-down menu in the upper-left corner of the Cloud Shell window.

First, I created a variable to store my **Application (client) ID** that I had recorded in Task 1. In the Cloud Shell, I ran:

```
$applicationId = 'Ecf8Q~paGwH9YghQXwZyGPDJshXbZPLL~UPbKbUF'
```

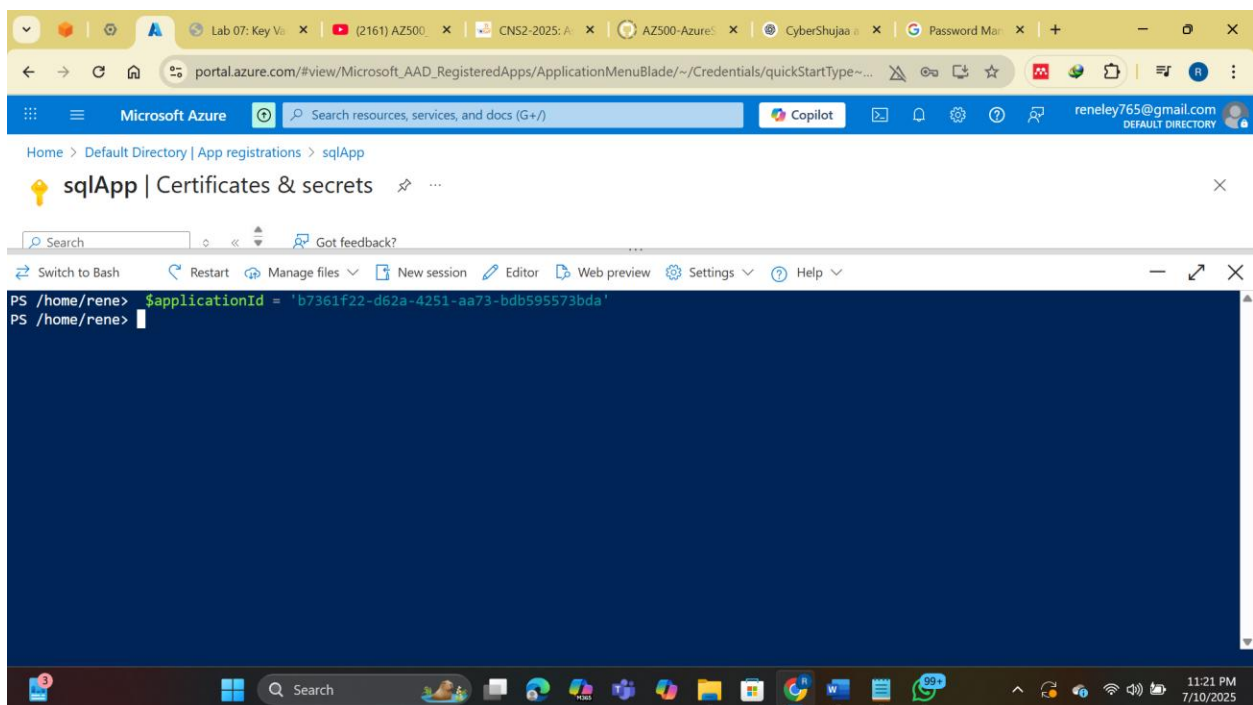


Fig 2.9 Creating a variable to store sqlapp id

Next, I needed to store the name of my Key Vault in a variable, so I ran:
`$kvName = (Get-AzKeyVault -ResourceGroupName 'AZ500LAB10').VaultName`
and confirmed it by outputting `$kvName` to verify that the Key Vault name was set correctly.

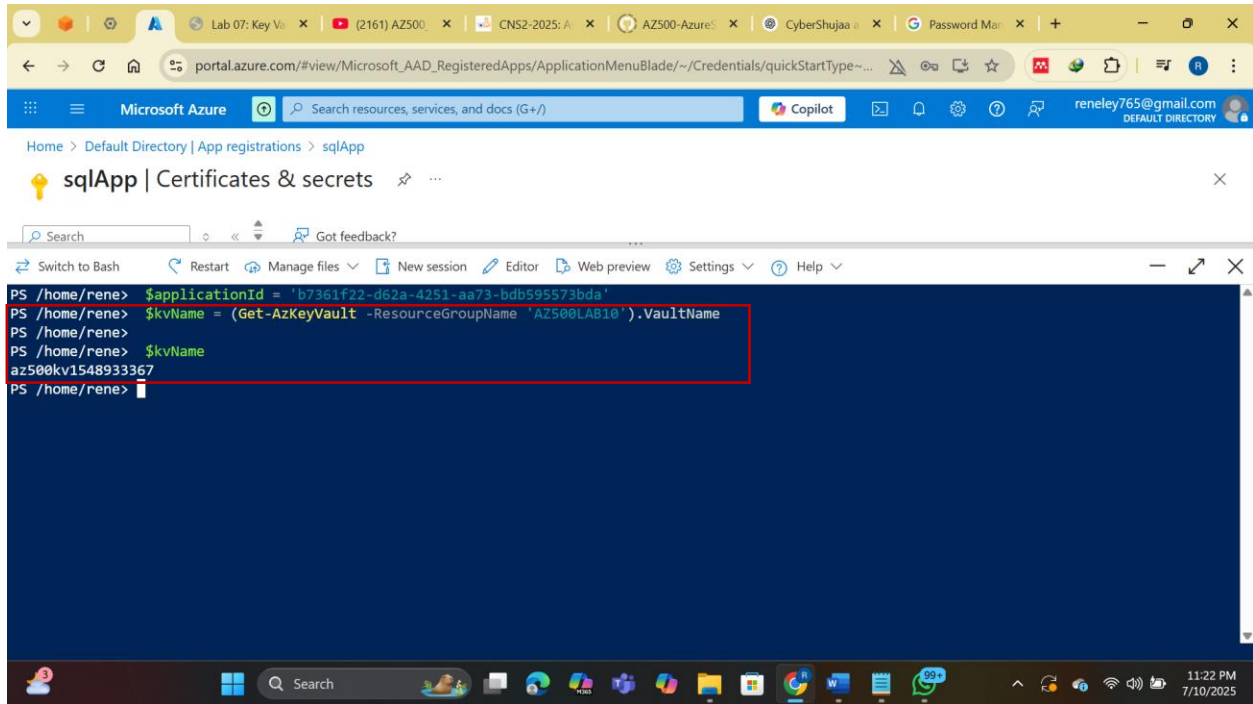


Fig 3.0 Storing the name of the key vault in a variable

Finally, I granted my registered application the necessary permissions on the Key Vault by running:
`Set-AZKeyVaultAccessPolicy -VaultName $kvName -ResourceGroupName AZ500LAB10 -ServicePrincipalName $applicationId -PermissionsToKeys get,wrapKey,unwrapKey,sign,verify,list`

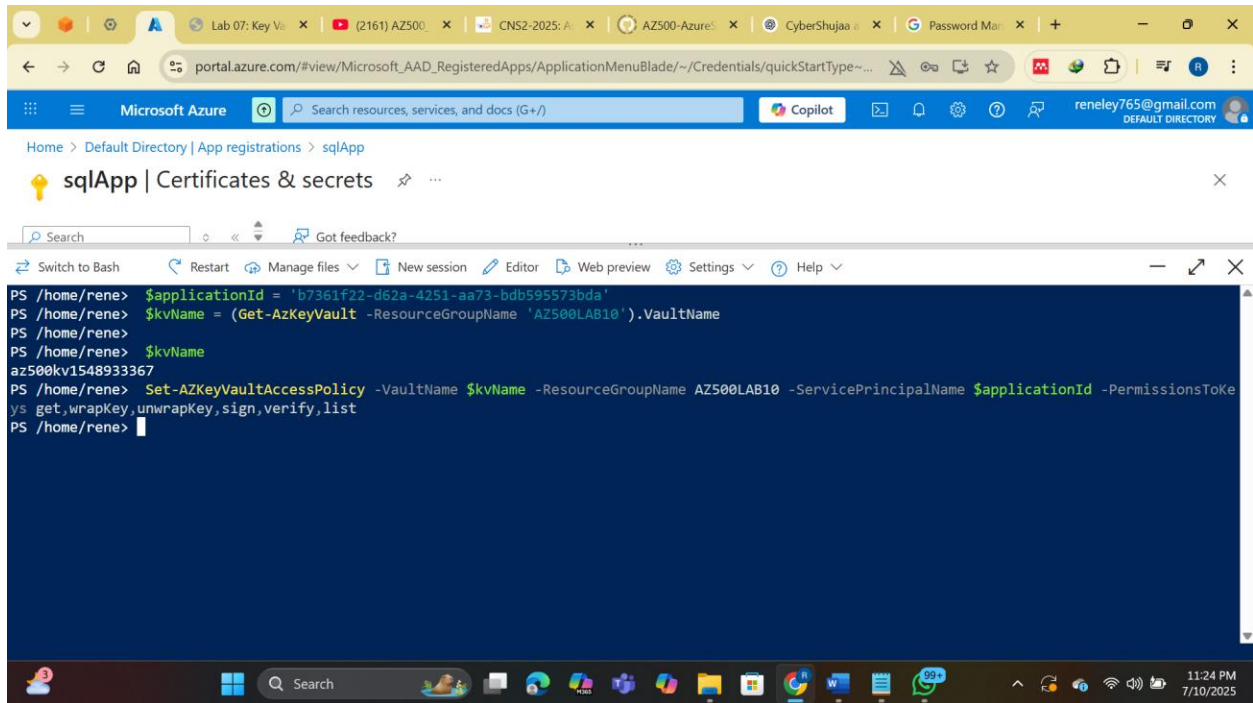


Fig 3.1 Granting the registered application necessary permission

This command gave my application permissions to **get, wrap, unwrap, sign, verify, and list** keys — which are the exact operations needed for secure interactions between the client application and the Key Vault.

Once the access policy was successfully applied, I closed the Cloud Shell pane. By completing this task, I ensured that my registered app now had explicit permissions to work with the keys stored in my Key Vault, which is critical for secure, encrypted operations with my Azure SQL Database.

Task 3: Retrieving SQL Azure Database ADO.NET Connection String

Next, I needed to get the **ADO.NET connection string** for my **Azure SQL Database** so I could connect my application securely. To do this, I opened the **Azure portal** and used the **Search resources, services, and docs** text box at the top of the page to search for **SQL databases**. After pressing **Enter**, I looked through the list and clicked on my database named **medical**.

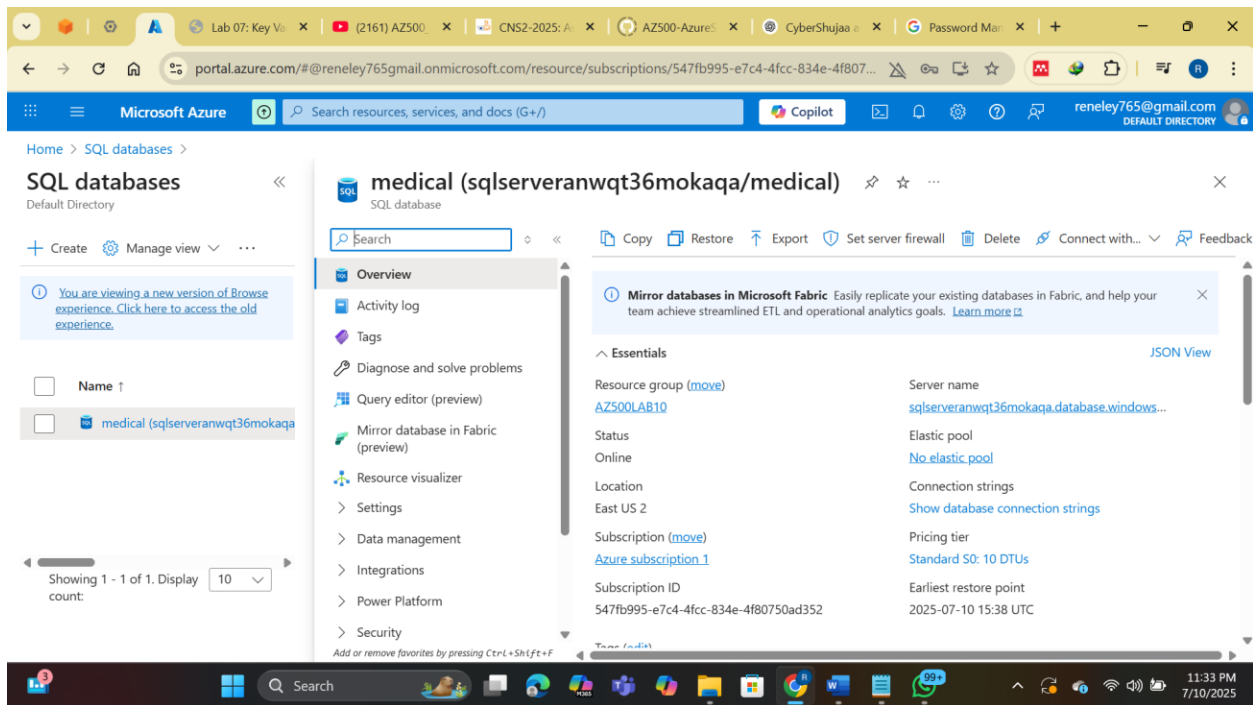


Fig 3.2 Navigating into the medical database

Once I was on the **SQL database blade**, I scrolled to the **Settings** section and clicked **Connection strings**. Here I specifically needed the **ADO.NET (SQL authentication)** connection string.

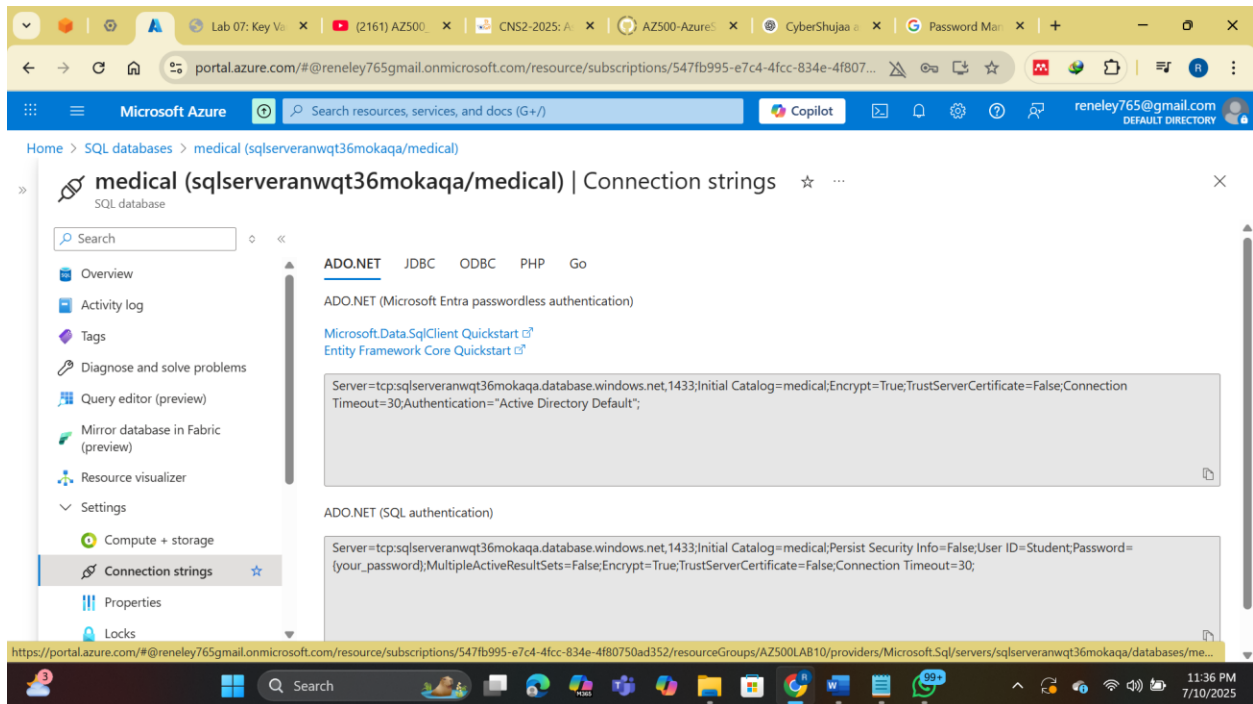


Fig 3.3 Connection strings blade

I recorded the **ADO.NET** connection string for later use. With the connection string ready, I now had the details needed to let my application securely connect to my Azure SQL Database using ADO.NET.

Task 4: Logging on to the Azure VM Running Visual Studio 2019 and SQL Management Studio 19

In the **Azure portal**, I searched for **Virtual machines** in the search bar at the top and pressed **Enter**. From the list of virtual machines, I located the VM that was deployed earlier. I clicked its name to open its blade and noted its public IP address.

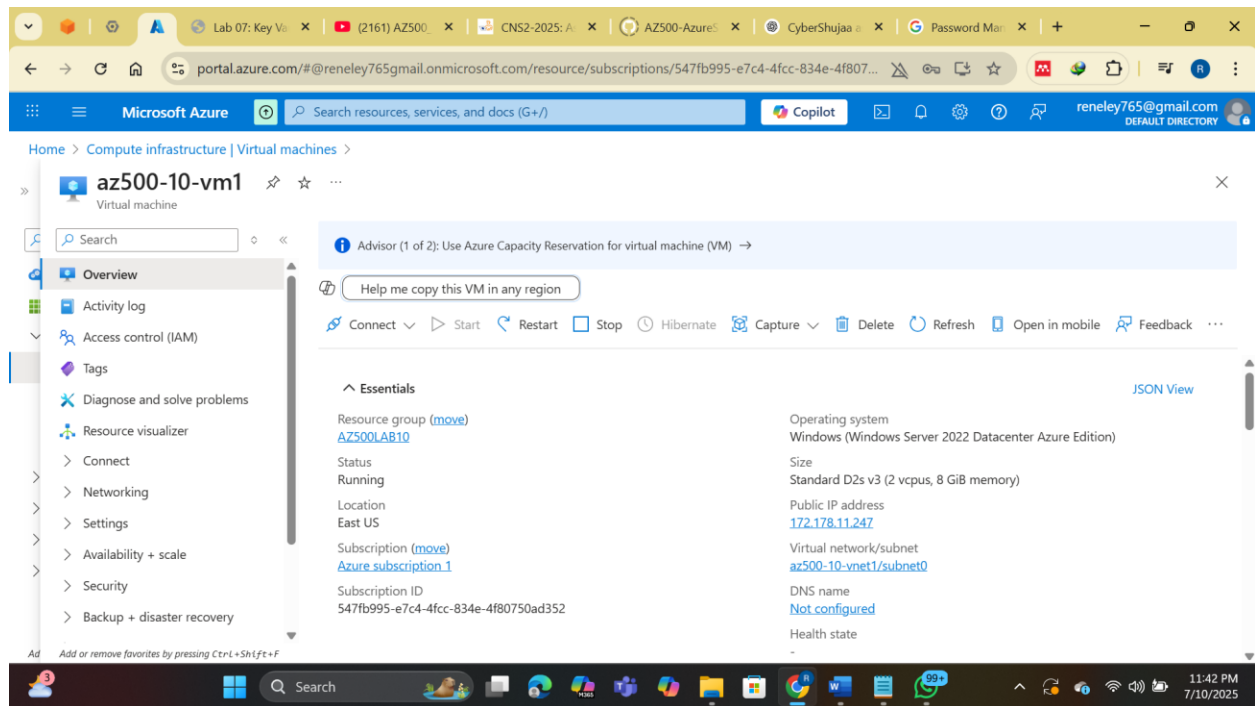


Fig 3.4 The earlier deployed vm

Task 5: Creating a Table in the SQL Database and Select Data Columns for Encryption

I began by navigating in the Azure portal to the blade of the medical SQL database, where I located the server name in the Essentials section and copied it to my clipboard for later use.

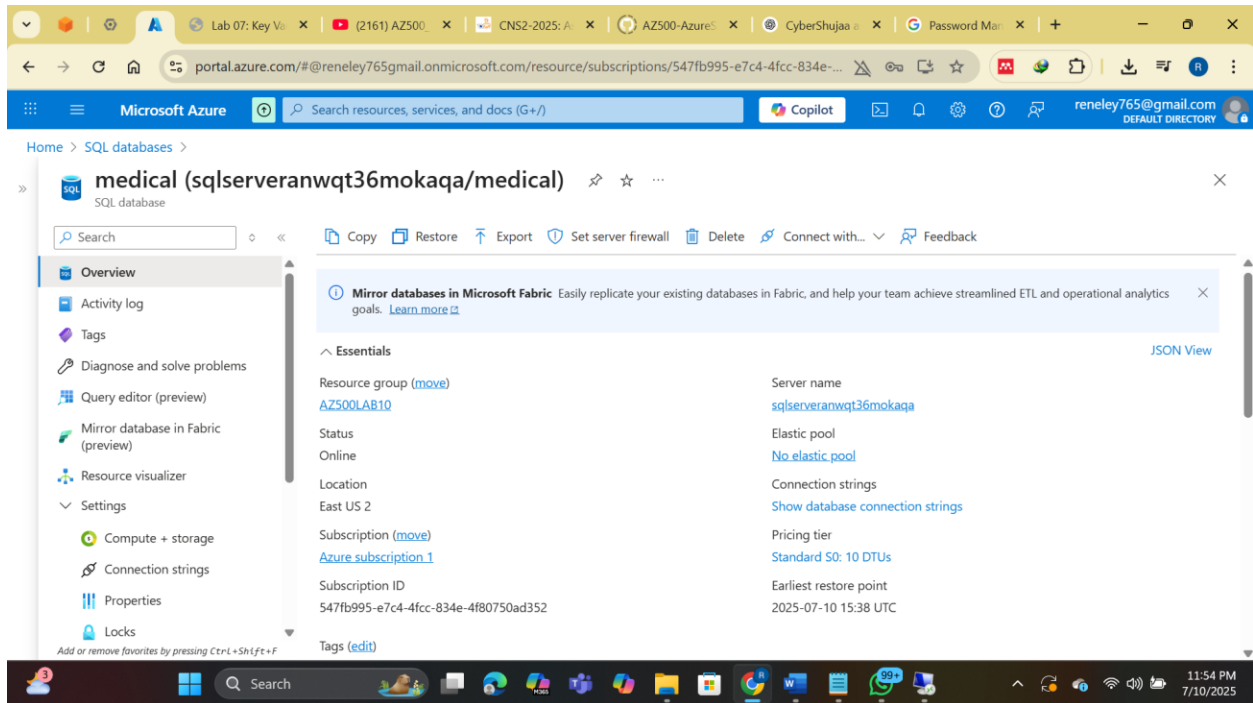


Fig 3.5 Sql blade(essentials part)

To allow connectivity from my Azure VM, I then clicked on Set server firewall in the toolbar. On the Firewall settings blade, I scrolled down, added a new firewall rule named "Allow Mgmt VM" using the public IP address of my az500-10-vm1 as both the start and end IP, saved the configuration, and confirmed that the firewall was now open for my management VM to access the SQL server. After configuring the firewall, I returned to the az500-10-vm1 blade, went to the Overview page, clicked Connect, and downloaded the RDP file to remotely connect to the VM.

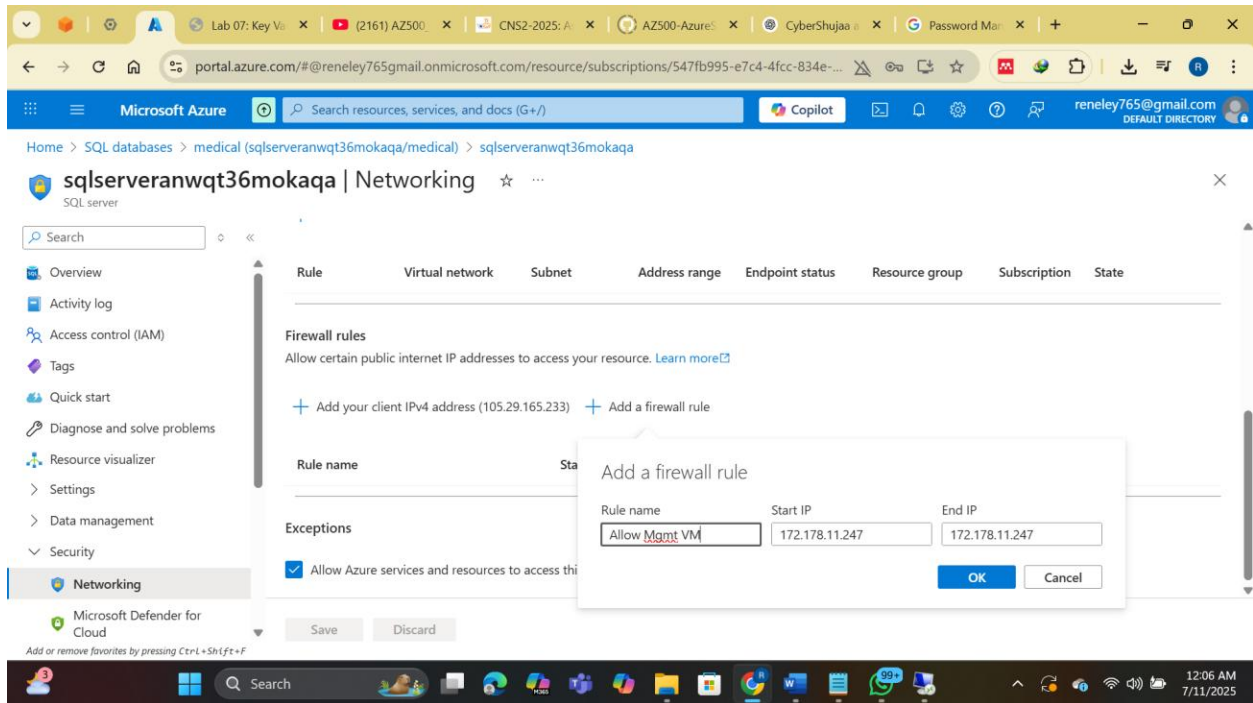


Fig 3.6 Setting up firewall rule

When prompted, I signed in using the Student account and my password created previously in Lab 02. Once connected via Remote Desktop and the Server Manager had loaded, I closed Server Manager to proceed.

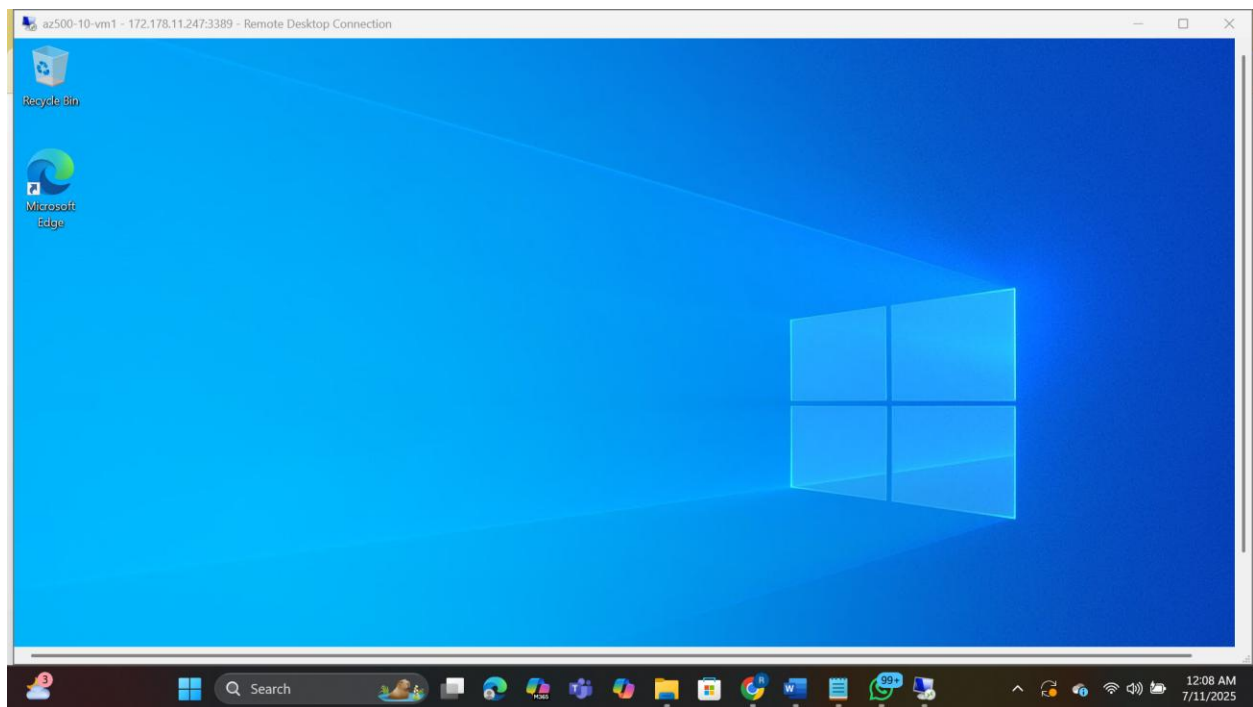


Fig 3.7 Rdping into the vm

I ensured that SQL Server Management Studio (SSMS) was installed on the VM, launched it, and in the Connect to Server dialog box, I selected Database Engine for the server type, entered the server name I had copied earlier, selected SQL Server Authentication, typed in the username Student and my lab password, and clicked Connect to establish the connection.

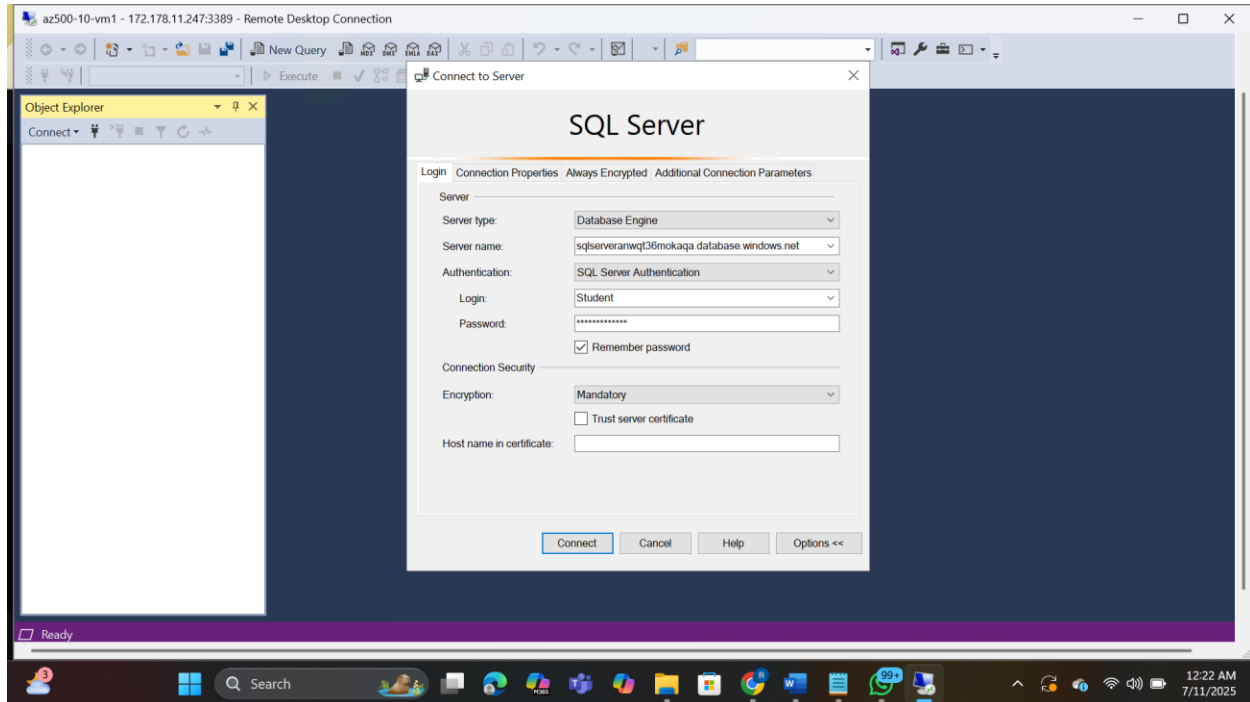


Fig 3.8 Signing in to SSMS

Inside SSMS, I expanded the Databases folder in the Object Explorer, right-clicked the medical database, and selected New Query. In the new query window, I pasted the provided SQL script to create the Patients table, which defined columns for PatientId, SSN, FirstName, LastName, MiddleName, StreetAddress, City, ZipCode, State, and BirthDate, and then executed the script to create the table successfully.

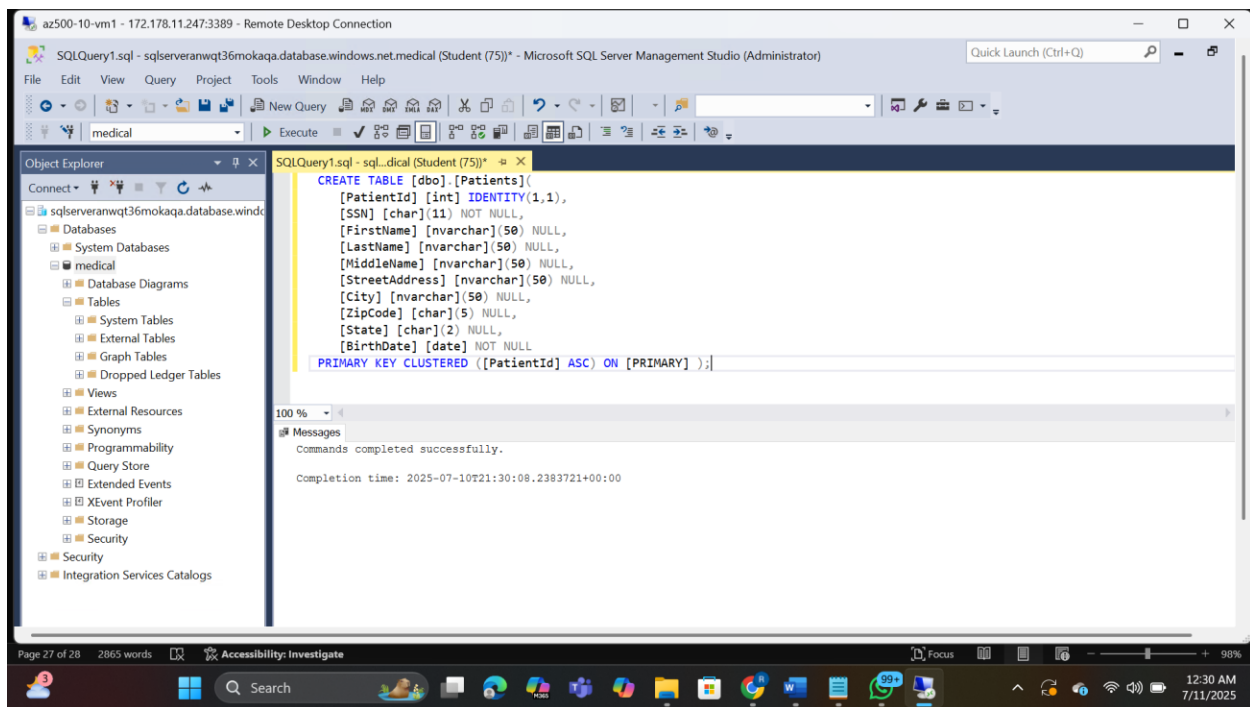


Fig 3.9 Pasting the script and executing

After the table was created, I expanded the medical database node, then the tables node, right-clicked `dbo.Patients`, and chose `Encrypt Columns` to launch the Always Encrypted wizard. I proceeded through the wizard by clicking `Next` on the Introduction page, then on the Column Selection page, I selected the `SSN` and `BirthDate` columns for encryption, setting `SSN` to `Deterministic` and `BirthDate` to `Randomized`, before clicking `Next`.

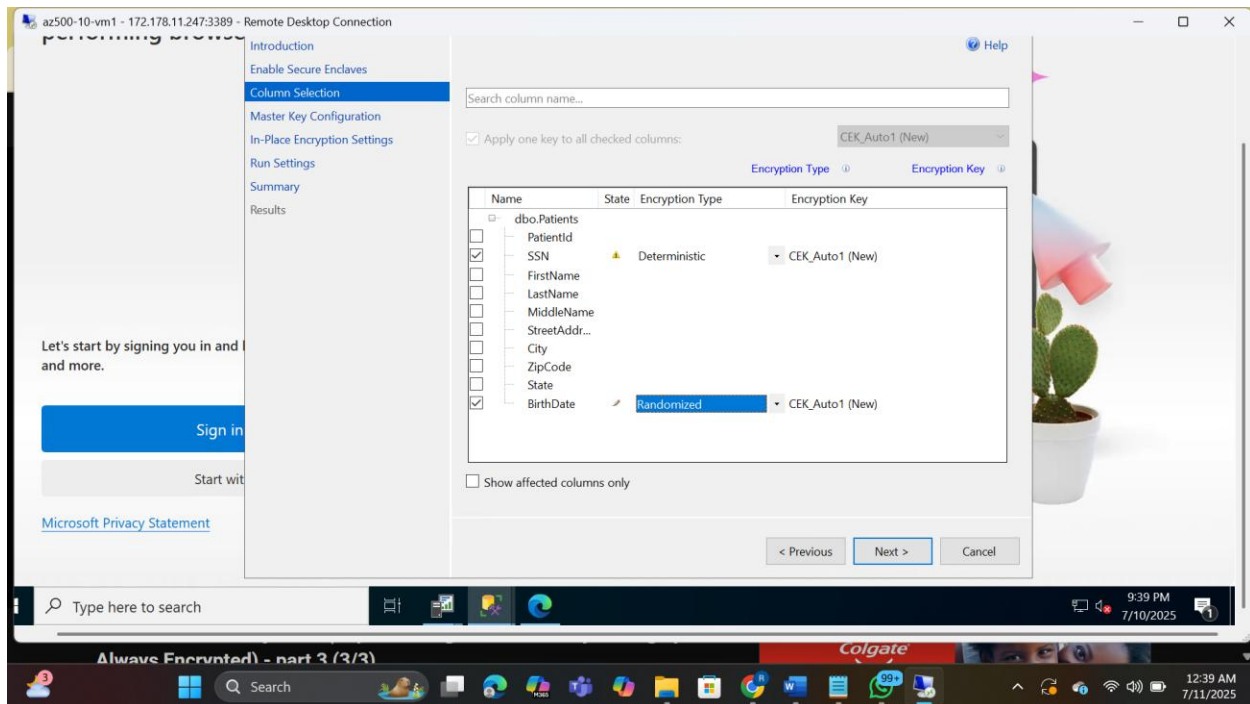


Fig 4.0 Selecting SSN and birth date to be encrypted

To sig in, I created a new student organizational email by signing into my Microsoft Entra ID tenant and using the the format studentname@reneley765.onmicrosoft.com. I set an initial password during creation, assigned the user the *Owner* role in my Azure subscription through Access Control (IAM) to grant full permissions, and verified the account by signing in to the Azure portal with the new credentials and completing the required password reset to activate the account for resource deployment. This is all because the email I had wasn't working (did not have the rights to perform actions I wanted to)

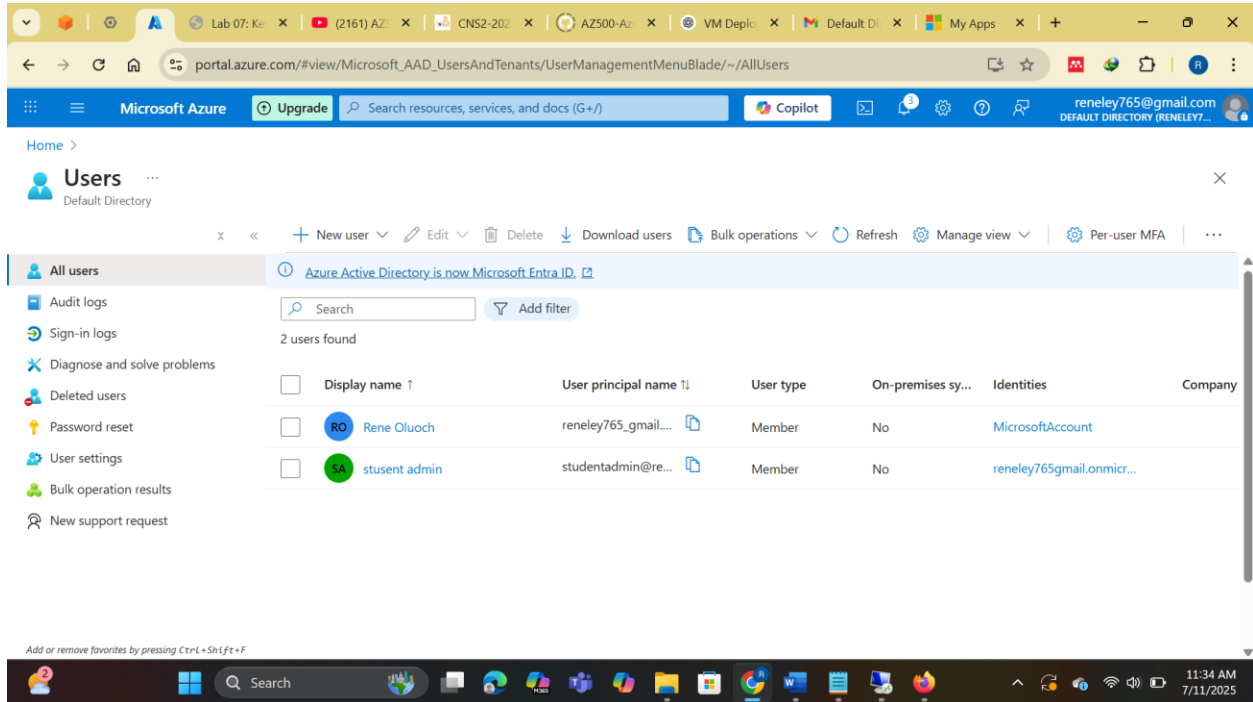


Fig 4.1 Creating student email

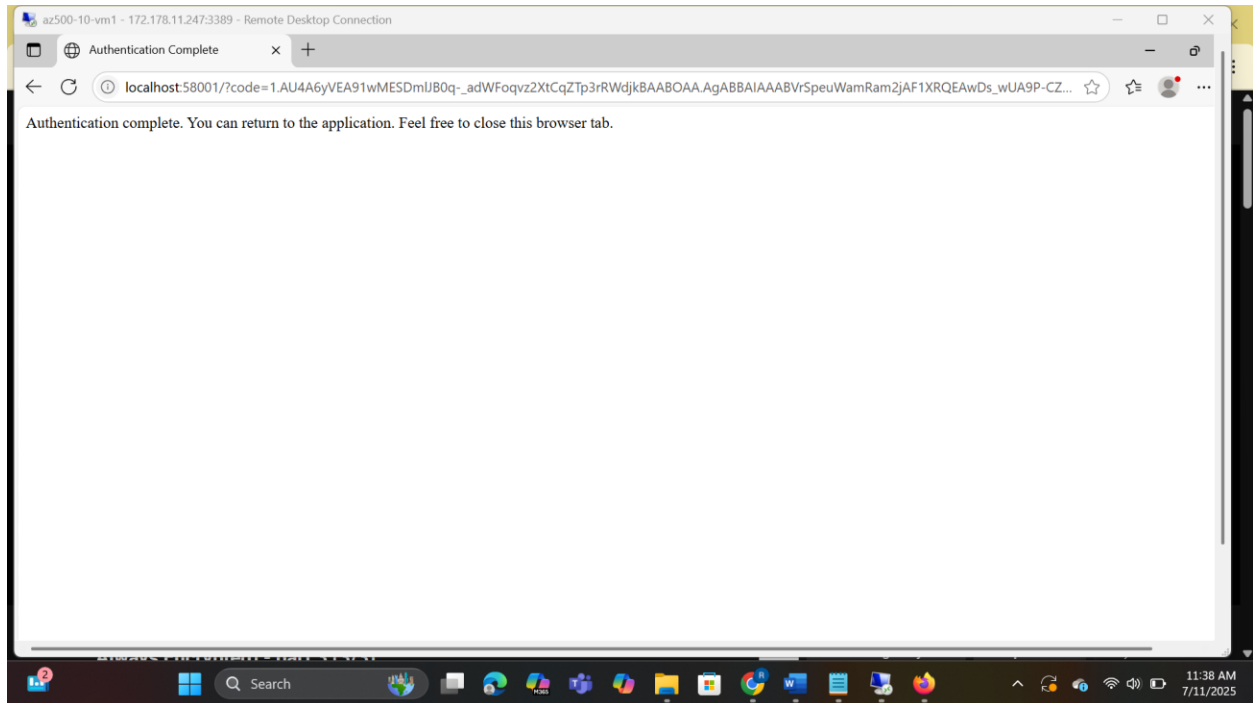


Fig 4.2 Successfully signing in

I verified in the Azure portal under the Key Vault's Access Policies that the Key Permissions for Rotation Policy Operations were unchecked, and under Privileged Key Operations the Release option was also unchecked, to avoid permission conflicts. Back in the wizard, on the Master Key Configuration page, I selected Azure Key Vault, signed in using the same account I used to provision the Key Vault, confirmed the correct Key Vault was available in the dropdown list, and clicked Next.

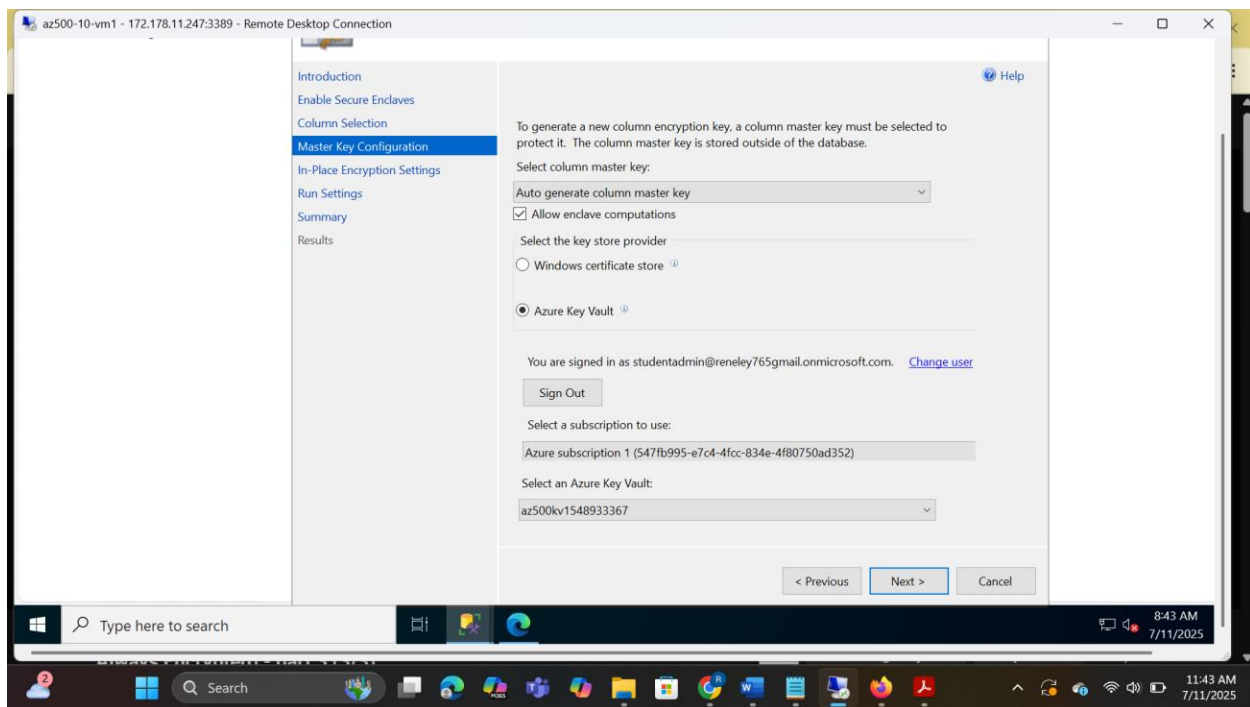


Fig 4.3 Selecting subscription group and key vault

I continued through the Run Settings page by clicking Next again, reviewed the Summary page, and clicked Finish to execute the encryption. When prompted, I signed in once more with the same account to finalize the configuration.

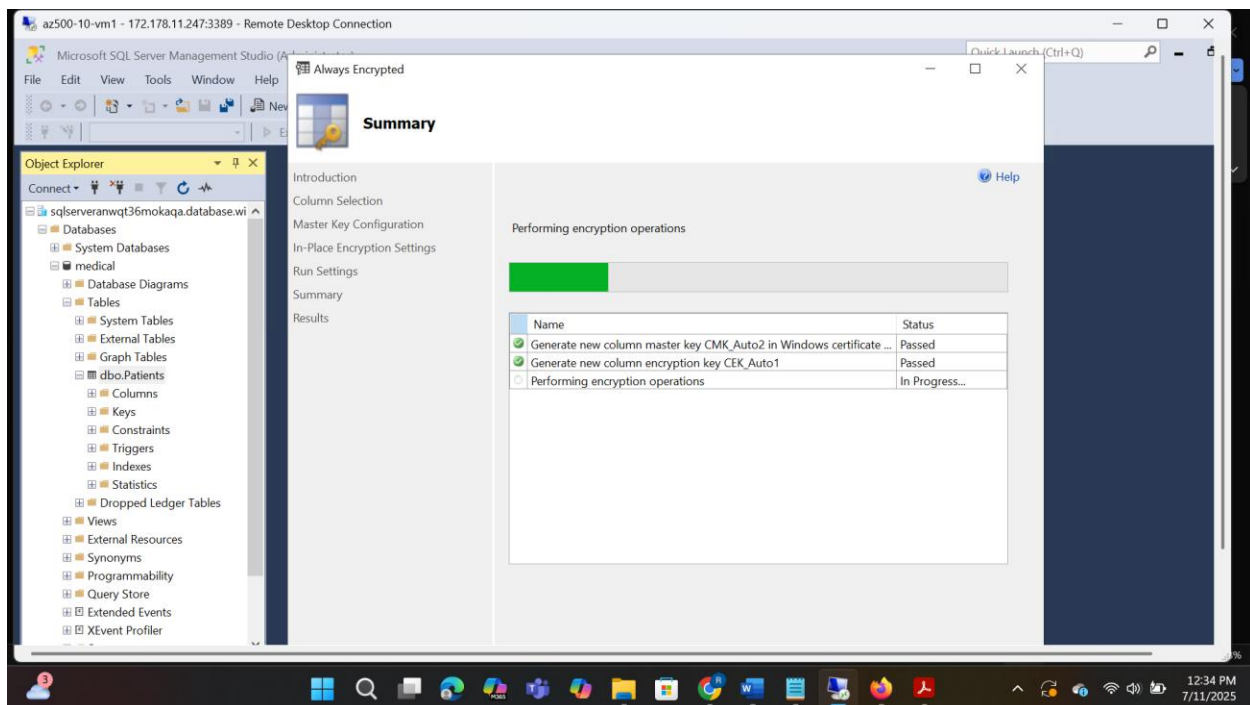


Fig 4.4 The encryption process

After the encryption process completed successfully, I closed the wizard on the Results page. Finally, within the SSMS Object Explorer, I verified under the medical database's Security node that the Always Encrypted Keys subnode appeared, containing both the Column Master Keys and Column Encryption Keys folders, confirming that the encryption keys were properly set up and linked to the Azure Key Vault for secure data operations.

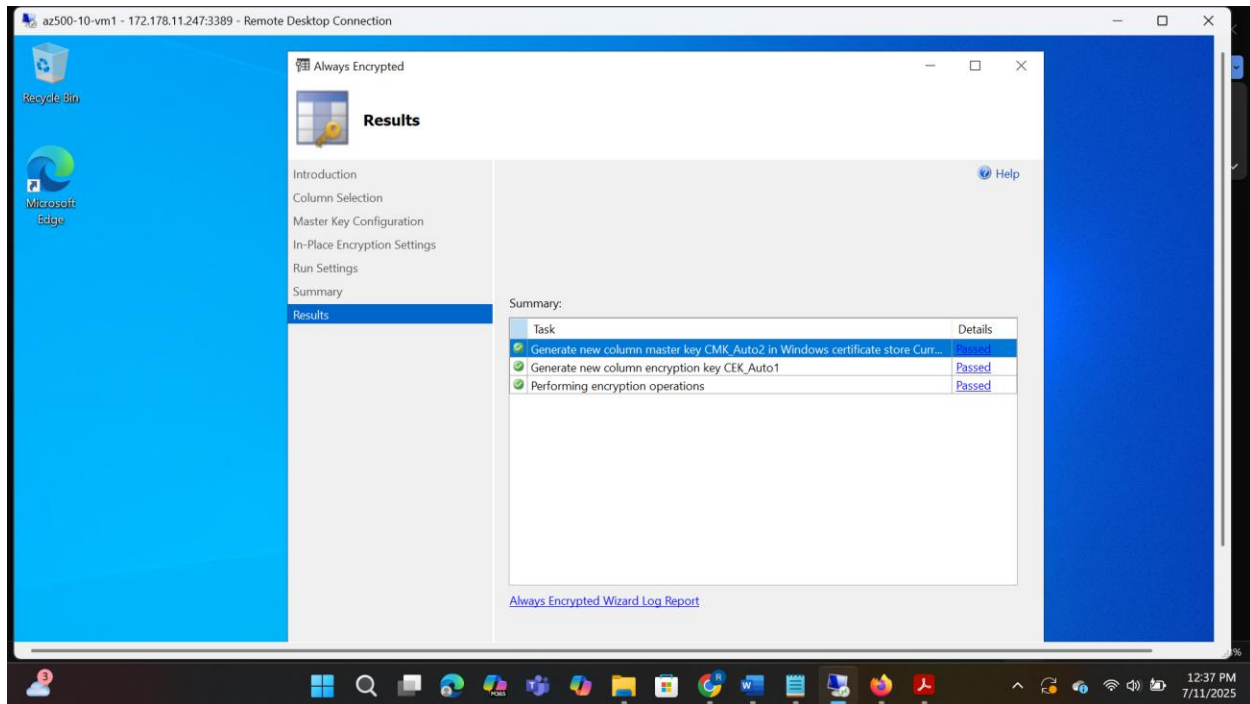


Fig 4.5 successful encryption

Exercise 4 : Demonstrating the use of Azure Key Vault in encrypting the Azure SQL database

Task 1: Running a data-driven application to demonstrate the use of Azure Key Vault in encrypting the Azure SQL database

Next, my goal was to demonstrate the practical use of Azure Key Vault in encrypting the Azure SQL database by running a simple data-driven console application. I started by launching Visual Studio 2022 on my az500-10-vm1 through the Remote Desktop session.

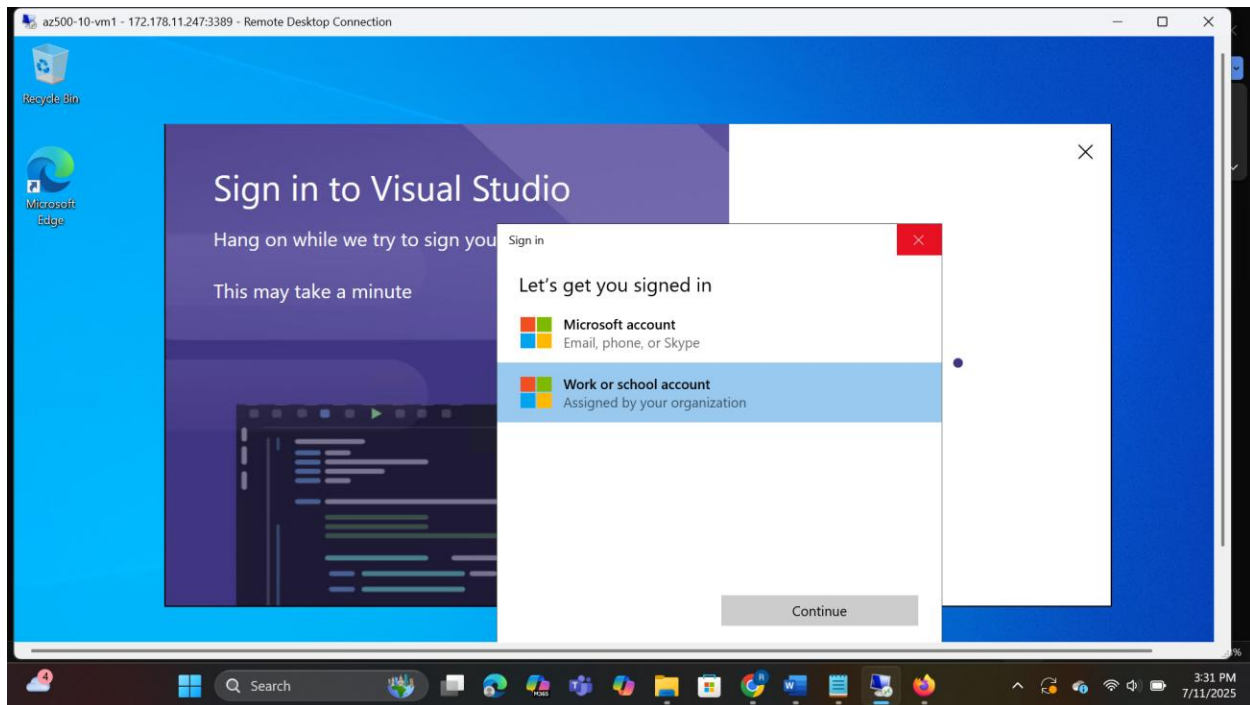


Fig 4.6 Launching vs code

When the Visual Studio welcome screen appeared, I signed in using the same credentials I used for my Azure subscription to ensure seamless access to all linked resources. On the Get Started page, I selected the option to create a new project and searched for the Console App (.NET Framework) template for C#. Once located, I clicked on it and proceeded to the next configuration screen. On the Configure Your New Project page, I named both the project and solution “OpsEncrypt” and made sure the target framework was set to .NET Framework 4.7.2 before creating the project.

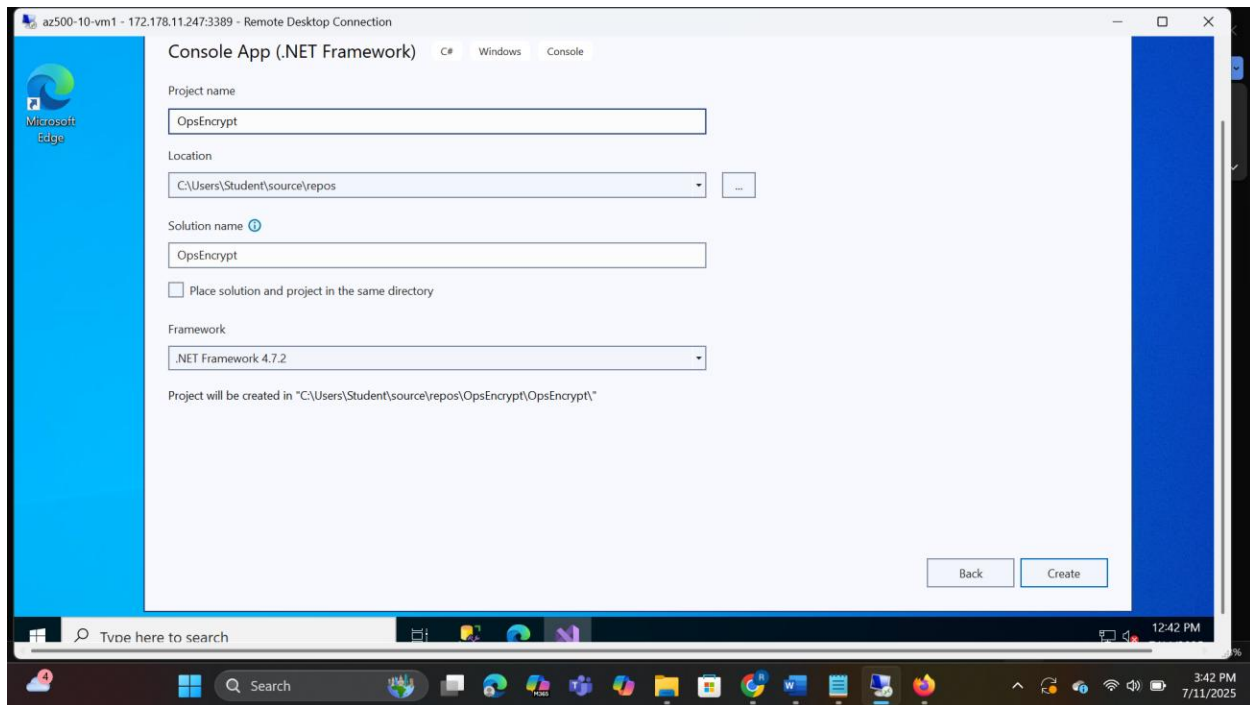


Fig 4.7 Creating the new project

After the project was created, I navigated to the Tools menu, opened the NuGet Package Manager, and launched the Package Manager Console. Inside the console, I installed the first required NuGet package by running Install-Package Microsoft.SqlServer.Management.AlwaysEncrypted.AzureKeyVaultProvider and then installed the second package by running Install-Package Microsoft.IdentityModel.Clients.ActiveDirectory.

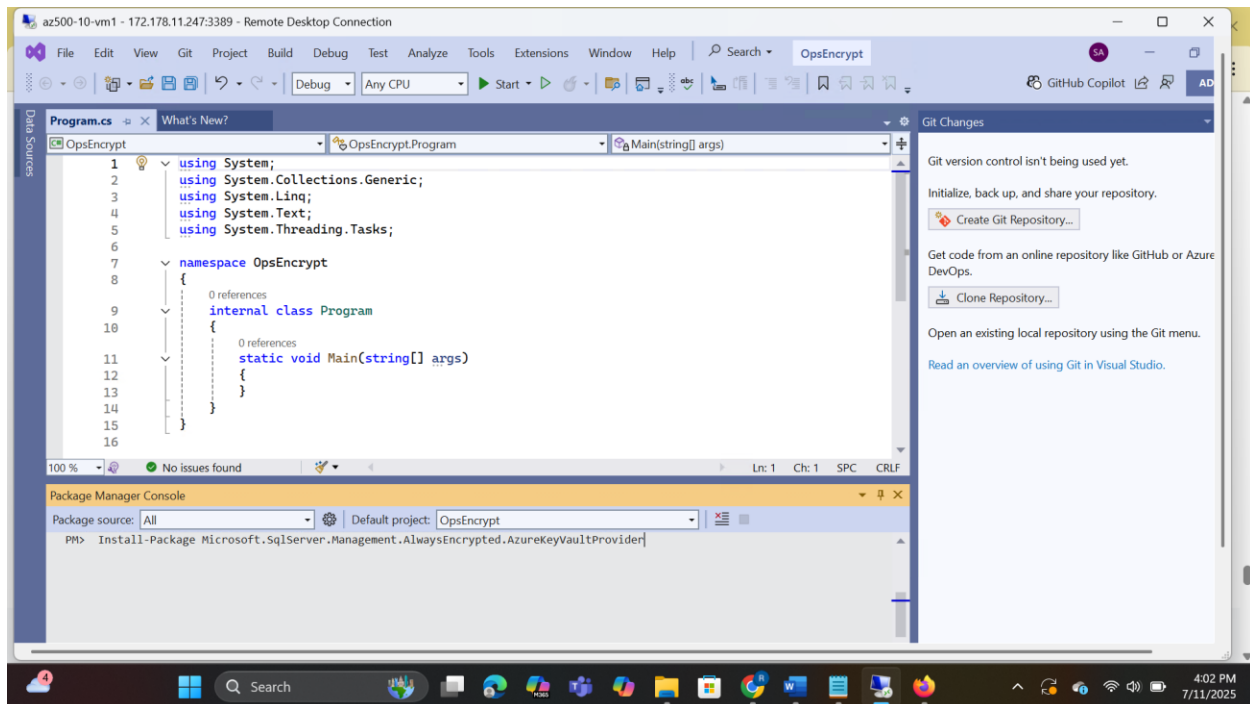


Fig 4.8 Installing required NuGet packages

With these dependencies added, I minimized the RDP session briefly to access the lab files on my local machine. I navigated to the provided path `\Allfiles\Labs\10\program.cs` and opened the `Program.cs` file in Notepad, copying all its contents to my clipboard. Returning to my RDP session, I went into the Solution Explorer of Visual Studio, located `Program.cs` under my `OpsEncrypt` project, opened it, and replaced its default content with the code I had copied.

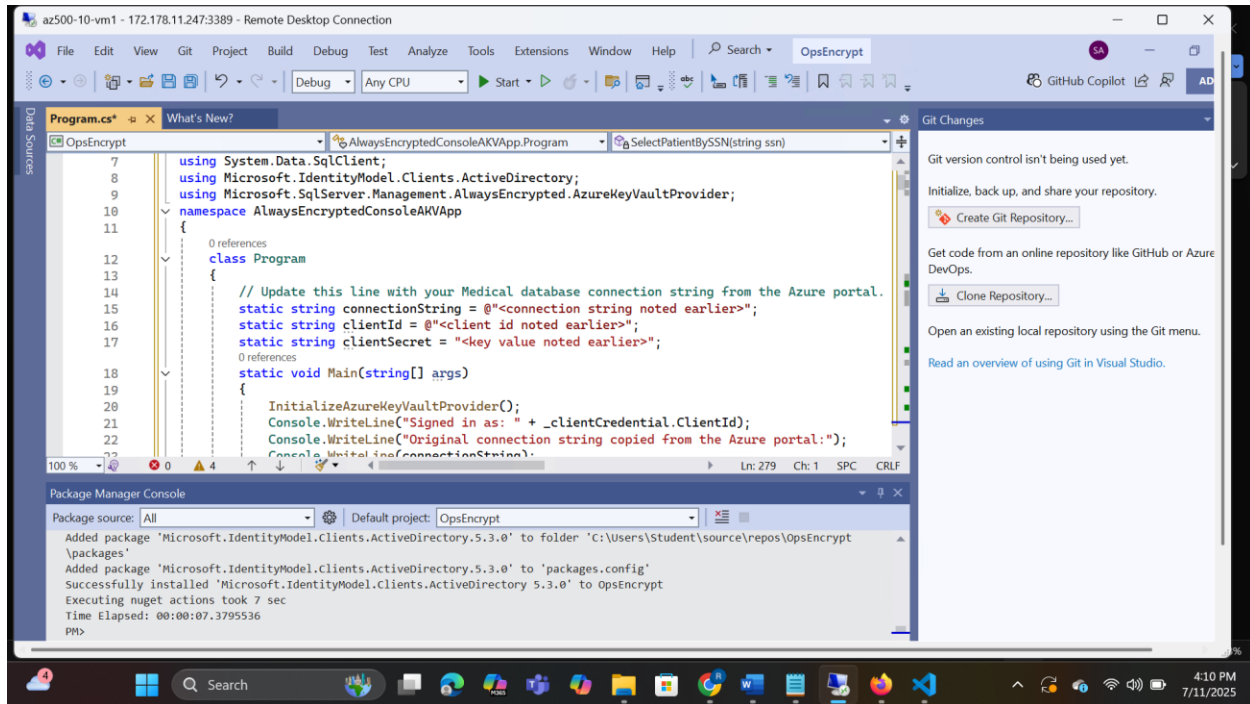


Fig 4.9 Pasting content of the program file

Next, I customized the placeholders in the code to match my environment. On line 15, I replaced `<connection string noted earlier>` with the ADO.NET connection string for my Azure SQL database that I had recorded earlier, making sure to replace the placeholder with the actual SQL Server password I set during the initial deployment in Exercise 1. On line 16, I replaced `<client id noted earlier>` with the Application (client) ID of the app registration I created in the lab, `b7361f22-d62a-4251-aa73-bdb595573bda`. On line 17, I replaced `<key value noted earlier>` with the value of `Key1` from the same app registration, `Ecf8Q~paGwH9YghQXwZyGPDJshXbZPLL~UPbKbUF`. After double-checking that all placeholders were accurately replaced, I clicked the `Start` button in Visual Studio to build and run the console application.

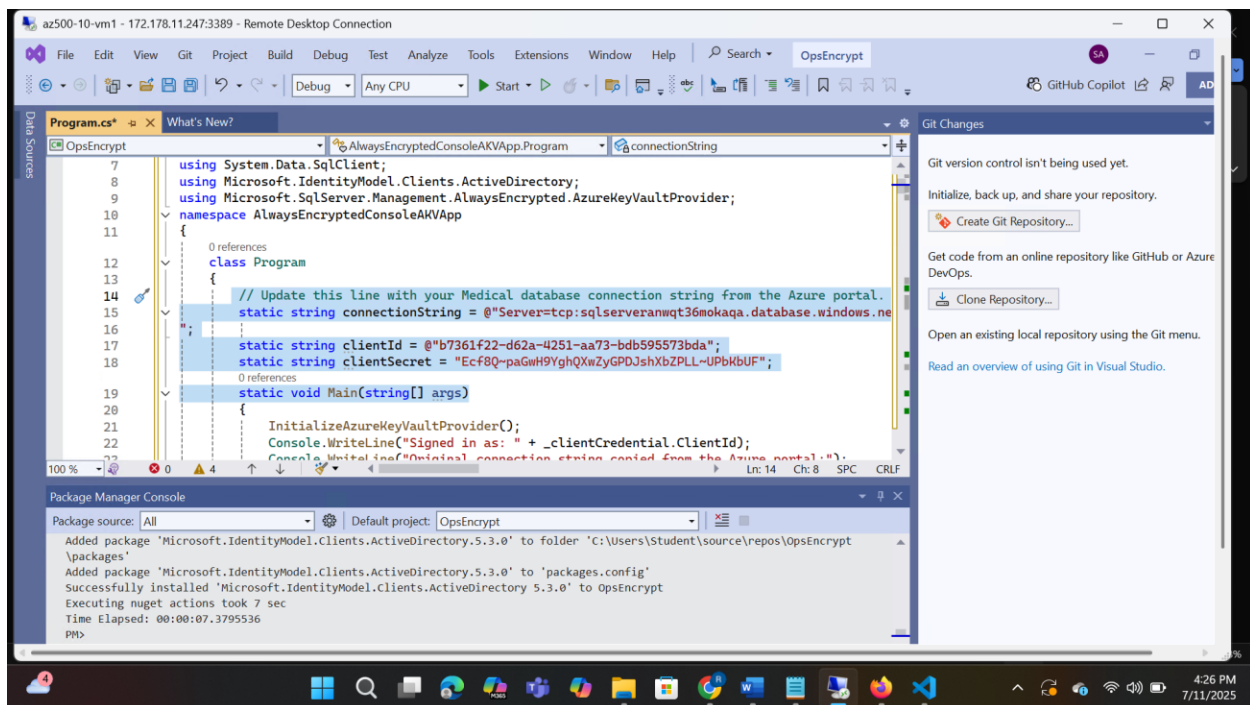


Fig 5.0 Updating the values with the earlier recorded values

This launched a Command Prompt window where the application began its operations and prompted me for the SQL database password, which I entered to allow the app to connect and insert data into the encrypted columns.

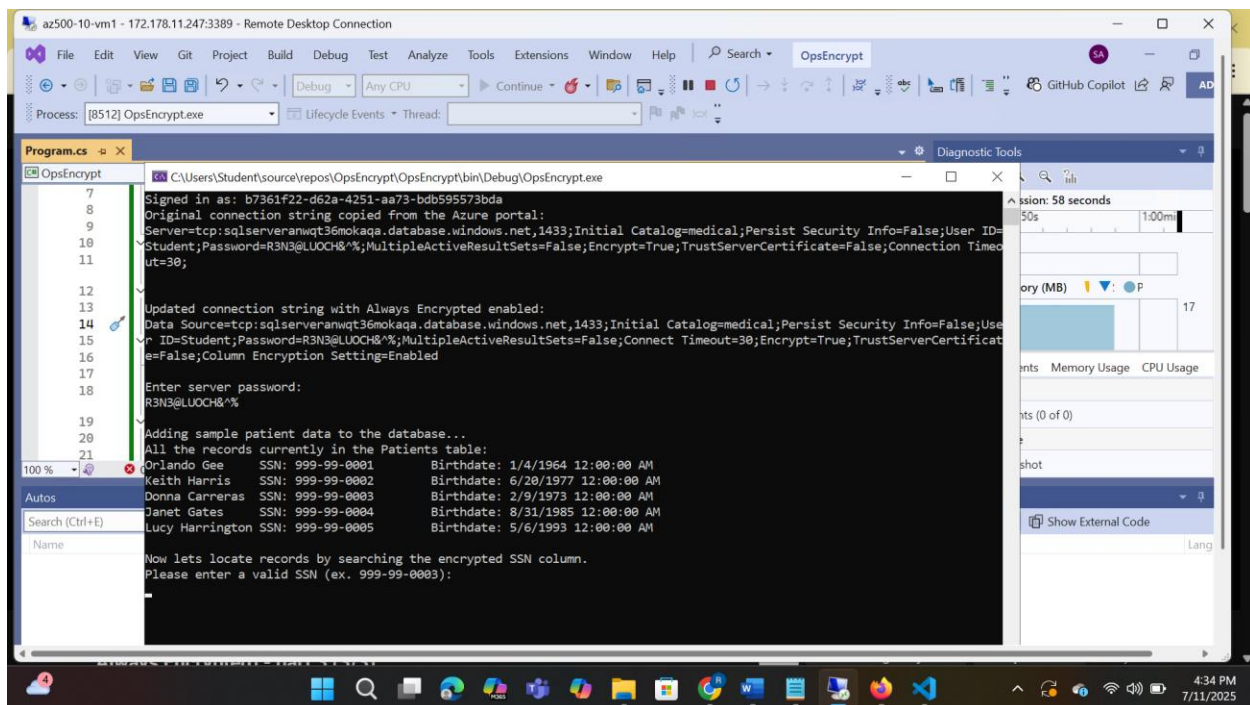


Fig 5.1 Inserting data into the encrypted colums

With the console app still running, I switched over to SQL Server Management Studio. In the Object Explorer, I right-clicked the medical database and opened a new query window. I ran a simple SELECT statement to verify that the new data was indeed being inserted and encrypted as intended: `SELECT FirstName, LastName, SSN, BirthDate FROM Patients;`. The query showed the encrypted data inside the SSN and BirthDate columns, confirming that the Always Encrypted configuration with the Azure Key Vault key was working properly.

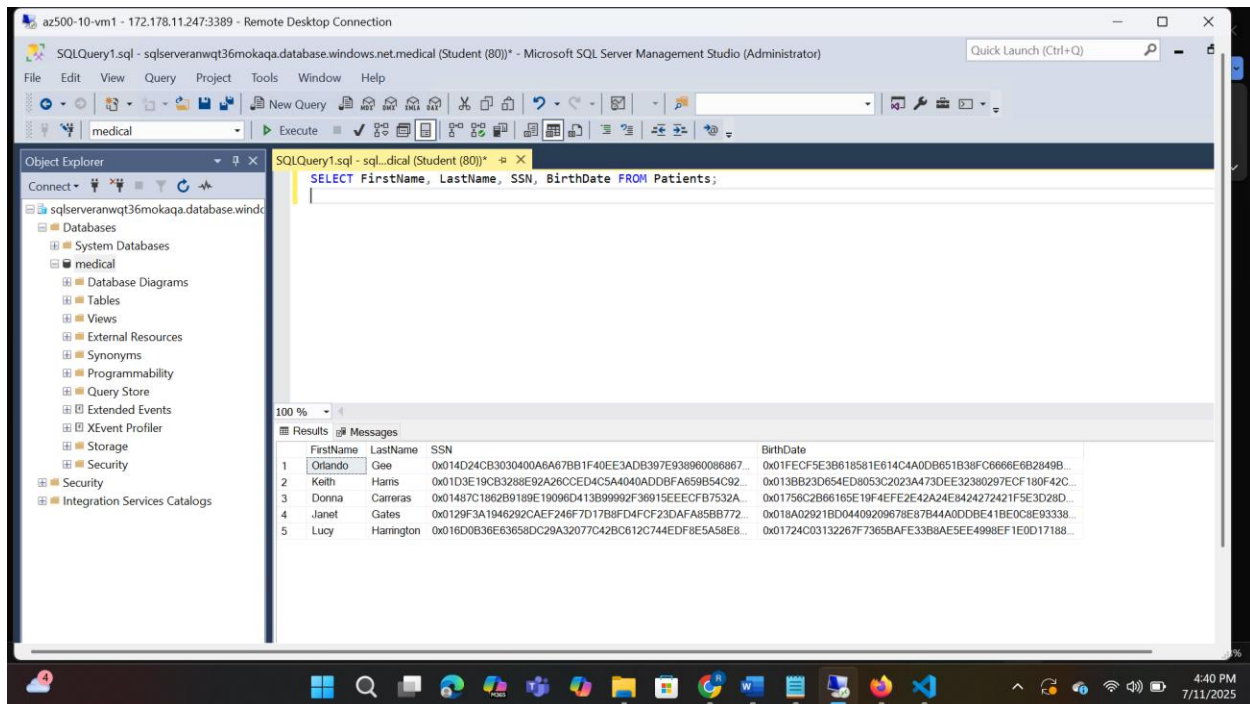


Fig 5.2 Keyvault working properly

Returning to the console application, I followed the prompt to input a valid SSN to test a read operation against the encrypted column. I typed 999-99-0003 and pressed Enter. The application successfully queried the encrypted SSN column and returned the correct decrypted result in plain text, demonstrating that the application could securely access and decrypt the protected data using the key stored in Azure Key Vault.

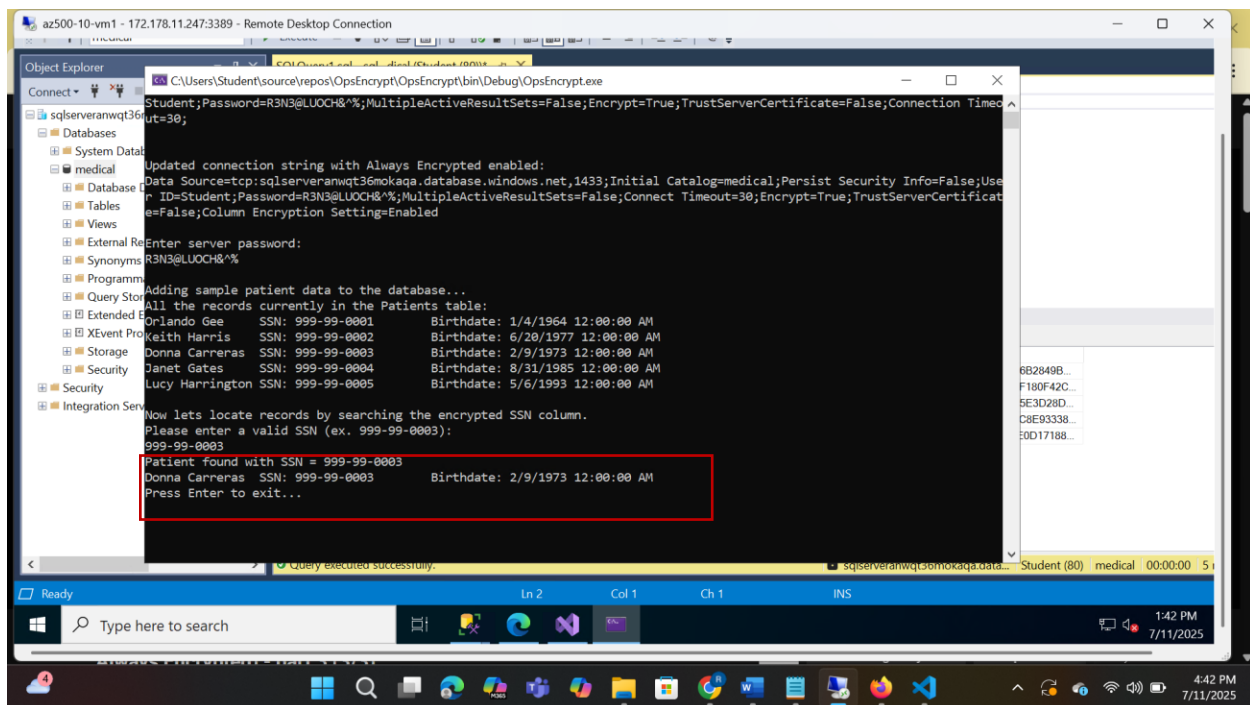


Fig 5.3 Verifying that the data returned by the query is not encrypted.

After verifying that the application worked end-to-end, I pressed Enter once more to terminate the console app session. To conclude this lab and avoid unnecessary charges, I cleaned up all the resources I had provisioned. Back in the Azure portal, I opened the Cloud Shell, ensured the session was running PowerShell, and when prompted, clicked Confirm. In the Cloud Shell pane, I executed `Remove-AzResourceGroup -Name "AZ500LAB10" -Force -AsJob` to remove the entire resource group along with all the resources associated with it. Once the command was accepted, I closed the Cloud Shell, confident that my environment was clean and no residual resources would generate additional costs.

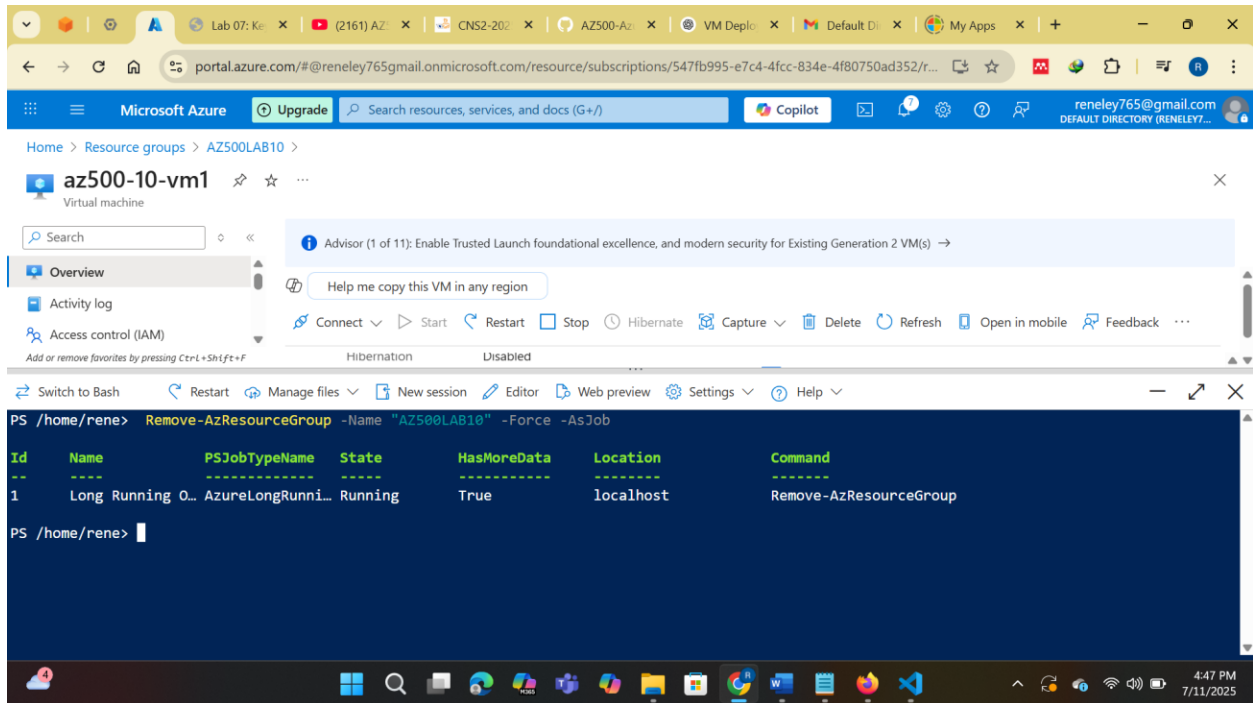


Fig 5.4 Deleting the resource

Conclusion

In conclusion, this lab demonstrated the practical use of Azure Key Vault to secure sensitive data within an Azure SQL Database by implementing Always Encrypted. I successfully configured firewall rules to allow secure connections, created a table with sensitive columns, and encrypted those columns using keys stored and managed securely in Azure Key Vault. I reinforced this setup by developing and running a data-driven console application in Visual Studio that securely inserted and retrieved encrypted data, showing how client applications interact with encrypted databases while the encryption keys remain protected in Azure Key Vault. This hands-on experience highlighted the importance of integrating key management services with database security to meet compliance and confidentiality requirements, and provided practical skills for safeguarding sensitive data in the cloud.